

FACULTAD DE CIENCIAS
GRADO EN MATEMÁTICAS
TRABAJO FIN DE GRADO
CURSO ACADÉMICO [2025-2026]

TÍTULO:

REDES NEURONALES CONVOLUCIONALES

AUTOR:

ÓSCAR CLIMENT IVORRA

Resumen

El presente trabajo tiene como objetivo realizar una introducción a las redes neuronales convolucionales y a su aplicación a la clasificación de imágenes.

En primer lugar, presentaremos la estructura de las redes neuronales prealimentadas con una única capa oculta, para posteriormente generalizar dicha estructura a las redes neuronales prealimentadas multicapa. En este contexto, explicaremos los métodos más comunes para entrenar una red neuronal, introduciendo la noción de función de error y presentando el método del descenso estocástico del gradiente y el método de retropropagación.

A continuación, presentaremos las redes neuronales convolucionales y explicaremos su relevancia para la clasificación de imágenes.

Ilustraremos la implementación de una red neuronal convolucional y sus ventajas respecto a arquitecturas de redes neuronales más genéricas, en un ejemplo de clasificación con 10 clases, utilizando el conjunto de datos MNIST, que contiene 70.000 imágenes de dígitos escritos a mano del 0 al 9.

Palabras clave: Red neuronal convolucional, red neuronal prealimentada, método de retropropagación, MNIST y función de error.

Abstract

This paper aims to provide an introduction to convolutional neural networks and their application to image classification.

First, we present the structure of feedforward neural networks with a single hidden layer, subsequently generalizing this architecture to multilayer feedforward neural networks. Within this framework, we explain the most common methods for training a neural network, introducing the concept of the loss function and presenting the stochastic gradient descent and backpropagation methods.

Next, we introduce convolutional neural networks and explain their relevance to image classification.

We illustrate the implementation of a convolutional neural network and its advantages over more generic neural network architectures through a 10-class classification example using the MNIST dataset, which contains 70,000 images of handwritten digits from 0 to 9.

Keywords: Convolutional neural network, feedforward neural network, backpropagation method, MNIST, loss function.

Índice

1. Introducción	5
2. Redes neuronales	7
2.1. Redes Neuronales Prealimentadas (FNN)	9
2.1.1. Función de error y entrenamiento de la red	12
2.2. Redes neuronales prealimentadas con múltiples capas ocultas	13
2.3. Entrenamiento de las redes neuronales	14
2.3.1. Descenso del gradiente	16
2.3.2. Descenso estocástico del gradiente	17
2.3.3. Descenso del gradiente por mini-lotes (mini-batch SGD)	18
2.3.4. Retropropagación	19
2.4. Ejemplo práctico (MNIST)	24
3. Redes Neuronales Convolucionales (CNN)	27
3.1. Detectores de características	29
3.2. Equivarianza a la traslación	31
3.3. Padding (relleno)	33
3.4. Convoluciones con paso (Stride)	35
3.5. Capas de agrupación (pooling layer)	36
3.6. Convoluciones multidimensionales	38
3.7. Ejemplo práctico de CNN (MNIST)	40
3.8. Comparación de los modelos implementados	42
4. Conclusión del trabajo	43
Referencias	43
A. Código de Python y figuras empleadas	46
B. Detalles del desarrollo del trabajo	46

1. Introducción

En muchas aplicaciones de interés, el objetivo es resolver un problema de predicción: dado un *input* (datos de entrada), x , se quiere predecir un *output* (salida) y . El *input* podría ser, por ejemplo, un vector que describe el funcionamiento de una máquina industrial (temperatura, presión, número de horas de uso, antigüedad, etc.) y el *output* la energía consumida durante su funcionamiento. Cuando el *output* que se quiere predecir es una variable cuantitativa, como en el ejemplo anterior, el problema recibe el nombre de **problema de regresión**. Cuando el *output* es una variable cualitativa, el problema recibe el nombre de **problema de clasificación** (binaria, si la variable que se quiere predecir puede tomar sólo dos valores, y multiclase si puede tomar tres o más valores). Un ejemplo de problema de clasificación binaria podría ser el problema en el cual se quiere clasificar una transferencia bancaria como fraudulenta o NO fraudulenta. En ese caso, el *input* consistiría en un vector que describe las características de la transferencia (importe, hora realizada, país de origen, etc.). Un problema de clasificación multiclase, por otro lado, podría ser, por ejemplo, el problema de predicción en el cual el *input* es un vector que describe el perfil de un paciente (edad, sexo, historial clínico, práctica de deportes, hábito de fumar, etc.) y el *output* que se quiere predecir es el estado de salud del paciente tras recibir una cierta intervención (variable cualitativa con tres niveles “bueno”, “malo” y “regular”).

La solución de los problemas de predicción planteados requiere definir **una función de predicción**, $f(\cdot)$, que asocia a cada *input* x un *output* $f(x)$.

El aprendizaje automático proporciona herramientas (algoritmos) para construir funciones de predicción con buenas propiedades y cuantificar el rendimiento de dichas funciones. Un algoritmo de aprendizaje automático funciona de la forma siguiente.

El punto de partida es un conjunto de N parejas (*input* x , *output* y). Dicho conjunto se divide de forma aleatoria en dos: un **conjunto de datos de entrenamiento**, de dimensión N_{train} , y un **conjunto test**, de dimensión N_{test} .

El algoritmo recibe como entrada el conjunto de entrenamiento. En el ejemplo anterior, sobre la clasificación de transferencias bancarias como fraudulenta o NO fraudulenta, el conjunto de datos de entrenamiento consistiría en un conjunto de N_{train} transferencias bancarias (*input*) y de la información para cada una de ellas de si es fraudulenta o NO fraudulenta (*output*). En el ejemplo sobre la energía consumida por las máquinas industriales, el conjunto de datos de entrenamiento consistiría en un conjunto de N_{train} máquinas industriales para las cuales se conocen las características (*input*) y la energía consumida

(*output*).

El algoritmo aprende de los datos de entrenamiento la relación entre *input* y *output* y proporciona como salida una función de predicción. Para evaluar el rendimiento de la función de predicción obtenida, se utiliza dicha función para predecir (a partir del *input*) la variable respuesta en el conjunto test y se calcula **el error de predicción** en el conjunto test, que será una función de la diferencia entre valores predichos y valores observados de la variable respuesta en el conjunto test.

El proceso de aprendizaje de la función de predicción a partir de los datos de entrenamiento recibe el nombre de aprendizaje supervisado.

En el ámbito del aprendizaje supervisado, las redes neuronales se han consolidado como una de las herramientas de mayor utilidad en las aplicaciones. Su estructura y funcionamiento están inspirados en la estructura y funcionamiento del cerebro humano, cuyas unidades de procesamiento son las neuronas, células eléctricamente activas. Como se muestra en la Figura 1, cada neurona está conectada con otras neuronas mediante conexiones llamadas sinapsis. Cuando una neurona “dispara” (se activa), envía un impulso eléctrico que pasa por el axón y de este llega a las sinapsis, que liberan señales químicas llamadas neurotransmisores, que pueden estimular o inhibir la activación de otras neuronas. En el cuerpo humano hay aproximadamente 90 mil millones de neuronas, que constituyen una red de conexiones muy compleja, formada por alrededor de 10^{14} sinapsis.

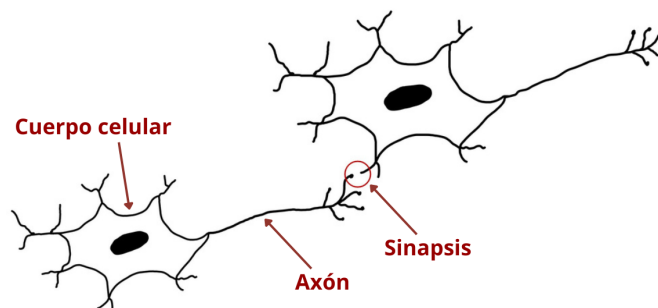


Figura 1: Ilustración de la conexión de dos neuronas del cerebro humano.

Si una neurona recibe una estimulación suficientemente alta de la activación de otras neuronas, entonces ella también se activa (envía un impulso eléctrico a otras neuronas). La medida en que la activación de una neurona puede inducir la activación de otra depende de la fuerza de la conexión que tiene con esta (es decir, de la sinapsis que las une). Durante el proceso de aprendizaje, la fuerza de algunas conexiones aumenta mientras que la de otras disminuye. En este sentido, los cambios en la fuerza de las conexiones entre neuronas representan un aspecto central del aprendizaje a partir de la experiencia.

Las redes neuronales son modelos matemáticos que imitan el funcionamiento y la interacción entre neuronas. En este trabajo nos centraremos en un tipo concreto de redes neuronales que recibe el nombre de redes convolucionales y que han sido diseñadas para problemas de procesamiento de imagen. En particular, en este trabajo nos centraremos en el uso de las redes convolucionales para problemas de clasificación (binarios o multiclase).

En la Sección 2, introduciremos los elementos básicos y el funcionamiento de las redes neuronales centrándonos, sobre todo, en la subclase de las redes neuronales prealimentadas, que contienen, como caso particular, las redes convolucionales.

En la Sección 3, presentaremos las redes convolucionales explicando con detalle su arquitectura y funcionamiento, ilustrando su aplicación y sus ventajas respecto a las redes neuronales estándar, en un problema real de clasificación multiclase.

Las conclusiones y algunas ideas de trabajo futuro se presentan en la Sección 4.

Las dos referencias más importantes en la preparación del trabajo son los capítulos 7, 8 y 10 de [1] y el capítulo 10 de [2].

2. Redes neuronales

Una red neuronal (NN, del inglés *Neural Network*) permite construir, a partir de un conjunto de datos de entrenamiento, una función de predicción no lineal que asocia a un *input* x un *output* y .

En muchas aplicaciones los datos de entrada (*input*) pueden ser muy complejos (documentos de texto, imágenes, grabaciones de sonido, etc.). Sin embargo, supondremos que, tras un preprocesamiento de los datos, el *input* se puede representar como un vector $\mathbf{X} = (X_1, X_2, \dots, X_p)$ de dimensión fija p , en $\mathbb{R}^p$.

Con respecto a la variable de salida, en un **problema de regresión** el *output* Y será un valor real, $Y \in \mathbb{R}$, mientras que en un **problema de clasificación** con C clases ($C = 2, 3, \dots$) el *output* será un vector numérico binario de dimensión C que representa la codificación 1-de- K (más conocida en inglés como *One-Hot Encoding*) de la variable cualitativa de interés. En base a esta codificación, un *output* que corresponde a una cierta clase $h \in \{1, 2, \dots, C\}$ es representado como un vector binario donde la componente h vale 1 y todas las demás valen cero.

En general, una red neuronal está formada por capas de neuronas (artificiales). La primera capa, que recibe el nombre de **capa de entrada**, como el mismo nombre sugiere, es la encargada de recibir los datos de entrada (el *input*) y contiene exactamente p uni-

dades (neuronas), siendo p la longitud del vector de *input*. Dichas unidades no realizan ningún procesamiento de los datos, simplemente almacenan los valores $X_1, X_2, \dots, X_p$ del vector *input*. Luego, una serie de capas intermedias, que se conocen como **capas ocultas**, procesan los datos de entrada. Finalmente, la última capa, que recibe el nombre de **capa de salida**, como el mismo nombre indica, devuelve el *output* y contiene un número de unidades igual a la longitud del vector de *output*, es decir, una única unidad, si se trata de un problema de regresión, o exactamente C unidades, si se trata de un problema de clasificación con C clases.

Las neuronas (unidades) de la red neuronal están conectadas por arcos dirigidos que representan el equivalente de las sinapsis del cerebro. Un arco dirigido desde la neurona i hasta la neurona j indica que la activación de la neurona i puede inducir la activación de la neurona j . A cada arco dirigido va asociado un **peso** que representa la fuerza de la conexión entre las neuronas.

La forma más sencilla de redes neuronales son las redes neuronales prealimentadas (FNN, del nombre en inglés *Feed-forward Neural Network*), en las cuales las neuronas de cada capa (a partir de la primera capa oculta) solo reciben información de la capa anterior, es decir, la información se mueve en una única dirección (hacia adelante) de la capa de entrada hasta la capa de salida pasando por las capas ocultas.

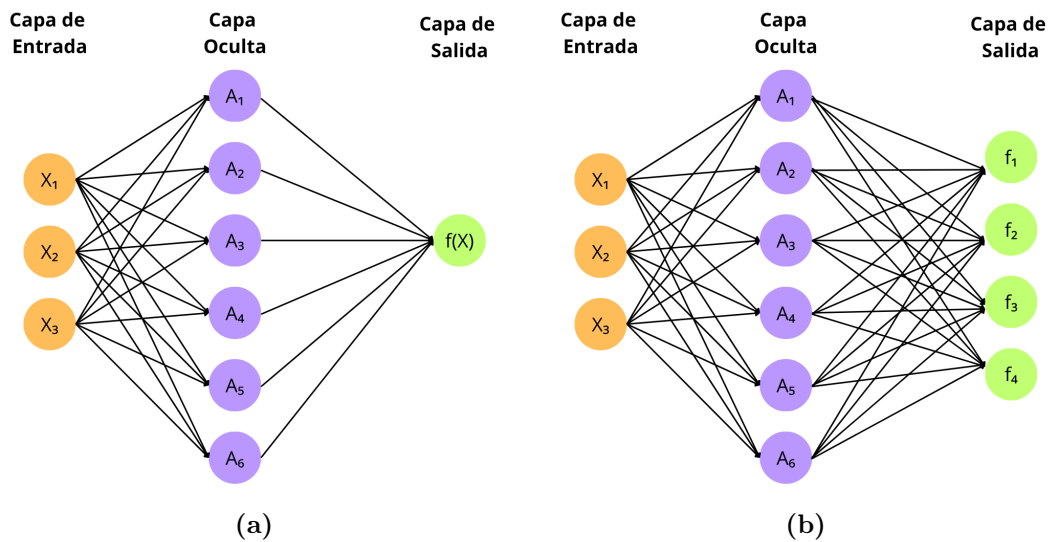


Figura 2: Ejemplos de redes neuronales prealimentadas (FNN) con una única capa oculta para los distintos tipos de problemas: (a) FNN para un problema de regresión, formado por una capa de entrada con 3 unidades, una capa oculta con 6 unidades y una capa de salida con una única unidad. (b) FNN para un problema de clasificación, formado por una capa de entrada con 3 unidades, una capa oculta con 6 unidades y una capa de salida con 4 unidades.

Un ejemplo de red neuronal prealimentada con una única capa oculta para un problema de regresión y un problema de clasificación se muestra en la Figura 2.

2.1. Redes Neuronales Prealimentadas (FNN)

Para describir el funcionamiento de las neuronas artificiales, que representan la unidad básica de procesamiento de la información de una FNN, consideremos en primer lugar el caso más sencillo en el cual hay una única capa oculta con K neuronas (que etiquetamos con los números $1, 2, \dots, K$) y una capa de salida con M neuronas, como se muestra en la Figura 3.

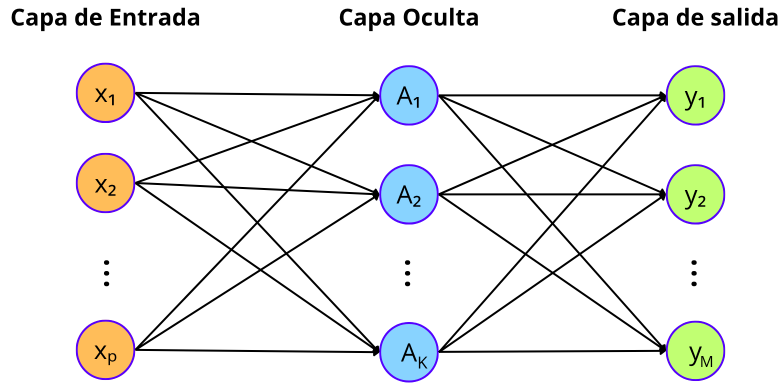


Figura 3: Ilustración de una red neuronal prealimentada (FNN) con una única capa oculta, formada por una capa de entrada con p unidades, una capa oculta con K unidades y una capa de salida con M unidades.

La neurona k de la capa oculta ($k \in \{1, \dots, K\}$) recibe una señal (estimulación) de las neuronas de la capa de entrada. La estimulación total recibida por la neurona k de las demás neuronas (de la capa de entrada), que denotaremos por a_k , recibe el nombre de **preactivación**, y es una combinación lineal de las señales X_j 's disparadas por las neuronas de la capa de entrada más un término constante llamado **sesgo**,

$$a_k = w_{k0} + \sum_{j=1}^p w_{kj} \cdot X_j \quad , \quad k = 1, \dots, K \quad (1)$$

donde w_{kj} es el **peso** asociado a la conexión entre la entrada j y la neurona k de la capa oculta que formaliza la fuerza de las sinapsis que une a las dos neuronas y w_{k0} es el **sesgo** de la neurona k que permite desplazar la preactivación de la neurona.

La **activación** de la neurona k , es decir, la señal emitida por la neurona k de la capa oculta en respuesta a la estimulación recibida por las neuronas de la capa de entrada, es

una función no lineal de la preactivación a_k y la denotaremos por A_k ,

$$A_k = g(a_k) \quad , \quad k = 1, \dots, K \quad (2)$$

donde $g(\cdot)$ es una función no lineal diferenciable que recibe el nombre de **función de activación** e imita el proceso de “disparo” de una neurona biológica. Un ejemplo de función de activación es la función *ReLU* (*Rectified Linear Unit*), que asocia a una preactivación a el valor cero si $a \leq 0$ y el mismo valor a si $a > 0$. Es decir, si la estimulación proveniente de las neuronas de la capa de entrada ($\sum_{j=1}^p w_{kj} \cdot X_j$) es menor o igual que una cierta cota ($-w_{k0}$), la neurona k de la capa oculta no “dispara” ninguna señal; mientras que si la estimulación proveniente de las neuronas de la capa de entrada es suficientemente fuerte ($\sum_{j=1}^p w_{kj} \cdot X_j > -w_{k0}$), la neurona k de la capa oculta “dispara” una señal $A_k = a_k$. Otras funciones de activación que se suelen utilizar para las neuronas de las capas intermedias se muestran en la Tabla 1.

Función de activación	Expresión matemática
Leaky ReLU	$g(x) = \begin{cases} 0,1x & \text{si } x \leq 0 \\ x & \text{si } x > 0 \end{cases}$
Función radial gaussiana	$\phi(x) = e^{-x^2}$
Tangente hiperbólica	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Binaria	$I_{(0,\infty)}(x) = \begin{cases} 0 & \text{si } x \leq 0 \\ 1 & \text{si } x > 0 \end{cases}$
Softmax $\sigma : \mathbb{R}^n \rightarrow [0, 1]^n$	$\sigma_i(x_1, \dots, x_n) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$

Tabla 1: Ejemplos de funciones de activación más utilizadas.

El funcionamiento de las neuronas de la capa de salida es parecido al funcionamiento de las neuronas de la capa oculta, con la única diferencia que, dichas neuronas reciben la información de las neuronas de la capa oculta, (y solo indirectamente de la capa de

entrada). La preactivación y la activación de la neurona h de la capa de salida, que denotaremos por a_h^{out} y A_h^{out} respectivamente, son dadas por

$$a_h^{out} = w_{h0}^{out} + \sum_{k=1}^K w_{hk}^{out} \cdot A_k \quad , \quad h = 1, \dots, M, \quad (3)$$

$$A_h^{out} = f(a_h^{out}) \quad , \quad h = 1, \dots, M, \quad (4)$$

donde w_{hk}^{out} es el peso asociado a la conexión entre la neurona k de la capa oculta y la neurona h de la capa de salida que formaliza la fuerza de las sinapsis que une a las dos neuronas, w_{h0}^{out} es el sesgo de la neurona h de la capa de salida y f es la función de activación de la neurona h .

Como función de activación de la neurona de la capa de salida para un problema de regresión (en el cual hay una única neurona en la capa de salida, $M=1$) suele utilizarse la función identidad (si la variable respuesta no es acotada) o la función *ReLU* si la variable respuesta es acotada inferiormente (u otros tipos de funciones que permitan preservar el rango de valores que puede tomar la variable respuesta). Obsérvese que aun utilizando la función identidad en la capa de salida, el *output* de la red, debido a la no linealidad de las funciones de activación de la capa oculta, es una función no lineal de los datos de entrada,

$$A_1^{out} = a_1^{out} = w_{10}^{out} + \sum_{k=1}^K w_{1k}^{out} \cdot A_k = w_{10}^{out} + \sum_{k=1}^K w_{1k}^{out} \cdot g(a_k) \quad (5)$$

Por otro lado, una elección común para la función de activación en un problema de clasificación con C clases (en el cual la capa de salida contiene exactamente C neuronas, $M=C$) es la función *Softmax*,

$$f(a_h^{out}) = \frac{e^{a_h^{out}}}{\sum_{r=1}^C e^{a_r^{out}}} \quad (6)$$

cuyos valores son no negativos y suman uno y, por lo tanto, pueden interpretarse como la probabilidad predicha por el algoritmo de que el individuo/objeto/imagen de interés pertenezca a una cierta clase.

Es importante observar que la notación A_h^{out} para la activación de las neuronas de la capa de salida (que devuelve el *output*) es en realidad una notación abreviada por $A_h^{out}(\mathbf{X}, \mathbf{w})$, ya que la activación de la neurona h de la capa de salida depende de los valores del vector de *input* $\mathbf{X} = (X_1, \dots, X_p)$ y del vector de parámetros asociados a la red, $\mathbf{w}$.

Para la red neuronal prealimentada en la Figura 3, dicho vector tiene longitud $d = (p + 1)K + (K + 1)M$ (que se obtiene sumando los Kp pesos que corresponden a las conexiones entre la capa de entrada y la primera capa oculta, los K sesgos asociados a las unidades de las capas ocultas, los KM pesos que corresponden a las conexiones entre la primera capa oculta y la capa de salida y los M sesgos asociados a las unidades de salida).

2.1.1. Función de error y entrenamiento de la red

Una vez especificada la arquitectura de la red neuronal (es decir, el número de unidades de la capa oculta (K), el número de unidades de la capa de salida (M), las funciones de activación de la capa intermedia y de la capa de salida), la FNN correspondiente puede interpretarse como un modelo de predicción paramétrico, con vector de parámetros $\mathbf{w}$, que asocia a cada *input* $\mathbf{X} = (X_1, \dots, X_p)$ un output $f(\mathbf{X}, \mathbf{w}) = (A_1^{out}(\mathbf{X}, \mathbf{w}), \dots, A_M^{out}(\mathbf{X}, \mathbf{w})) \in \mathbb{R}^M$.

Para un cierto valor del vector de parámetros $\mathbf{w}$, definimos el **error de entrenamiento** de la FNN, como la suma del error de predicción de la FNN en el conjunto de entrenamiento,

$$R(\mathbf{w}) = \sum_{i=1}^{N_{train}} L(f(\mathbf{X}_i, \mathbf{w}), Y_i) \quad (7)$$

donde $\mathbf{X}_i = (x_{i1}, \dots, x_{ip})$, Y_i y $f(\mathbf{X}_i, \mathbf{w})$ representan el vector *input*, el valor verdadero de la variable respuesta y el valor predicho de la variable respuesta para el i -ésimo individuo en el conjunto de entrenamiento, y L representa una **función de error** (también conocida como función de pérdida) que cuantifica el error de predicción para dicho individuo.

La estimación del vector de parámetros $\mathbf{w}$ se obtiene minimizando el **error de entrenamiento**, $R(\mathbf{w})$, es decir, resolviendo el problema de optimización

$$(P) \quad \min_{\mathbf{w}} R(\mathbf{w}) \quad (8)$$

Para problemas de regresión, la función de error más utilizada es la **pérdida cuadrática**,

$$L(f(\mathbf{X}_i, \mathbf{w}), Y_i) = (f(\mathbf{X}_i, \mathbf{w}) - Y_i)^2 \quad (9)$$

y la estimación del vector de parámetros $\mathbf{w}$ se obtiene resolviendo el problema de optimización,

$$(P) \quad \min_{\mathbf{w}} \sum_{i=1}^{N_{train}} (Y_i - f(\mathbf{X}_i, \mathbf{w}))^2 \quad (10)$$

Para problemas de clasificación con C clases, la función de error más utilizada es la **entropía cruzada**,

$$L(f(\mathbf{X}_i, \mathbf{w}), Y_i) = - \sum_{c=1}^C Y_{ic} \cdot \log(f_c(\mathbf{X}_i, \mathbf{w})) \quad (11)$$

donde $(Y_{i1}, Y_{i2}, \dots, Y_{iC})$ es la 1-de-K codificación de la variable respuesta para el i -ésimo individuo en el conjunto de entrenamiento y $f_c(\mathbf{X}_i, \mathbf{w})$ es la activación de la neurona c de la capa de salida que puede interpretarse como la probabilidad predicha por la FNN de que el individuo i -ésimo pertenezca a la clase c , $c \in \{1, 2, \dots, C\}$. En ese caso, la estimación del vector de parámetros $\mathbf{w}$ se obtiene resolviendo el siguiente problema de optimización,

$$(P) \quad \min_{\mathbf{w}} - \sum_{i=1}^{N_{train}} \sum_{c=1}^C [Y_{ic} \cdot \log(f_c(X_i, \mathbf{w}))] \quad (12)$$

La solución del problema de optimización en (8) se describe en detalle en la Sección [2.3](#).

2.2. Redes neuronales prealimentadas con múltiples capas ocultas

La red neuronal prealimentada con una única capa oculta descrita en la sección anterior, se puede extender de forma sencilla al caso de múltiples capas ocultas. Esto requiere introducir una notación adicional para distinguir entre las capas ocultas. Consideramos una red neuronal prealimentada con $L \geq 2$ capas ocultas. Denotaremos la preactivación y la activación de las neuronas de la primera capa oculta por

$$a_k^{(1)} = w_{k0}^{(1)} + \sum_{j=1}^p w_{kj}^{(1)} \cdot X_j \quad , \quad k = 1, \dots, K_1, \quad (13)$$

$$A_k^{(1)} = g_1(a_k^{(1)}) \quad , \quad k = 1, \dots, K_1, \quad (14)$$

donde $g_1(\cdot)$ es una función no lineal y diferenciable.

En la segunda capa oculta, el *input* no son los datos de entrada directamente, sino las activaciones de la primera capa oculta. En particular, denotaremos la preactivación y la

activación de las neuronas de la segunda capa por

$$a_k^{(2)} = w_{k0}^{(2)} + \sum_{j=1}^{K_1} w_{kj}^{(2)} \cdot A_j^{(1)} \quad , \quad k = 1, \dots, K_2, \quad (15)$$

$$A_k^{(2)} = g_2(a_k^{(2)}) \quad , \quad k = 1, \dots, K_2, \quad (16)$$

donde $g_2(\cdot)$ es una función no lineal diferenciable.

Y en general, denotaremos la preactivación y la activación de las neuronas de la capa oculta l -ésima, $l \in \{2, \dots, L\}$, como

$$a_k^{(l)} = w_{k0}^{(l)} + \sum_{j=1}^{K_{l-1}} w_{kj}^{(l)} \cdot A_j^{(l-1)} \quad , \quad k = 1, \dots, K_l \quad (17)$$

$$A_k^{(l)} = g_l(a_k^{(l)}) \quad , \quad k = 1, \dots, K_l \quad . \quad (18)$$

Por último, la preactivación y la activación de la neurona h de la capa de salida vienen dadas por:

$$a_h^{out} = w_{h0}^{out} + \sum_{k=1}^{K_L} w_{hk}^{out} \cdot A_k^{(L)} \quad , \quad h = 1, \dots, M \quad (19)$$

$$y_h^{out} = A_h^{out} = f(a_h^{out}) \quad , \quad h = 1, \dots, M \quad (20)$$

Además, para la red neuronal prealimentada multicapa, el vector de parámetros asociado a la red, $\mathbf{w}$, suponiendo que cada neurona de una capa está conectada a todas las neuronas de la capa siguiente, tiene longitud $d = \sum_{l=1}^L (K_{l-1} + 1)K_l + (K_L + 1)M$, donde $K_0 = p$.

2.3. Entrenamiento de las redes neuronales

Una vez definida la arquitectura de una red neuronal prealimentada (es decir, una vez especificado el modelo de predicción que se quiere utilizar), el paso siguiente consiste en determinar el vector de parámetros del modelo de red neuronal especificado. Dicho vector, que denotaremos por $\mathbf{w}$, incluye los pesos y los sesgos asociados a la red neuronal.

Como se ha comentado anteriormente, la estimación del vector de pesos $\mathbf{w}$ se realiza minimizando el error de entrenamiento, $R(\mathbf{w})$, es decir, resolviendo el problema de optimización,

$$(P) \quad \min_{\mathbf{w}} \quad R(\mathbf{w}) \quad . \quad (21)$$

La optimización del error de entrenamiento, $R(\mathbf{w})$, puede realizarse de forma eficiente utilizando la información del gradiente $\nabla R(\mathbf{w})$, es decir, calculando las derivadas del error de entrenamiento con respecto a cada uno de los parámetros del modelo. En este sentido, las funciones de activación y las funciones de error (que miden el error de predicción) de uso más común en las aplicaciones, han sido elegidas, entre otras cosas, para asegurar que tanto la función de predicción $f(\mathbf{X}, \mathbf{w})$ como el error de entrenamiento $R(\mathbf{w})$ sean, por construcción, diferenciables.

El valor de $\mathbf{w}$ que minimiza el error de entrenamiento, $R(\mathbf{w})$, debe ser tal que

$$\nabla R(\mathbf{w}) = 0 \quad (22)$$

ya que, de no ser así, sería posible reducir el valor del error de entrenamiento, moviéndose en el espacio del parámetro en la dirección $-\nabla R(\mathbf{w})$, es decir, en la dirección opuesta a la dirección de máximo crecimiento, definiendo $\tilde{\mathbf{w}}$ como

$$\tilde{\mathbf{w}} = \mathbf{w} - \eta \nabla R(\mathbf{w}) \quad (23)$$

que para η suficientemente pequeño, proporciona $R(\tilde{\mathbf{w}}) < R(\mathbf{w})$. Las soluciones de la ecuación $\nabla R(\mathbf{w}) = 0$ reciben el nombre de **puntos estacionarios** y se pueden clasificar en máximos, mínimos y puntos de silla.

Desafortunadamente, la no linealidad de las funciones de activación, que se traduce en una dependencia altamente no lineal del error de entrenamiento de los pesos, implica que habrá muchos puntos del espacio del vector de parámetros, en los cuales el gradiente se anula (o es aproximadamente cero).

En particular, puede haber puntos estacionarios que corresponden a mínimos locales equivalentes (es decir, con el mismo valor de $R(\mathbf{w})$) así como puntos estacionarios que corresponden a mínimos locales no equivalentes (es decir, puntos de mínimos locales que proporcionan diferentes valores del error de entrenamiento $R(\mathbf{w})$).

Por lo tanto, a pesar de que el objetivo final es calcular el mínimo global del error de entrenamiento, la solución del problema de optimización en (21) podría no ser única, y sobre todo, métodos que utilizan la información del gradiente, podrían quedar atrapados en mínimos locales que proporcionan una aproximación muy pobre de la solución del problema de optimización en (21).

Afortunadamente, en la práctica, los algoritmos que utilizan la información del gradiente parecen funcionar relativamente bien. Algunos de los algoritmos de uso más común se describen en las Secciones 2.3.1-2.3.3.

Un método muy eficiente para el cálculo del gradiente del error de entrenamiento, conocido como **retropropagación**, se describe en la Sección 2.3.4.

2.3.1. Descenso del gradiente

Como se ha observado anteriormente, si el error de entrenamiento, $R(\mathbf{w})$, es diferenciable, siempre que el gradiente de $R(\mathbf{w})$ sea distinto de cero es posible reducir el valor de $R(\mathbf{w})$ moviéndose en el espacio del parámetro de $\mathbf{w}$ en dirección $-\nabla R(\mathbf{w})$. Esta es exactamente la idea del método del descenso del gradiente (GD) que funciona de la forma siguiente.

Después de haber elegido al azar un valor inicial $\mathbf{w}^{(0)}$ para el vector de parámetros $\mathbf{w}$, se va actualizando el valor de $\mathbf{w}$ mediante el siguiente proceso iterativo

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta_\tau \nabla R(\mathbf{w}^{(\tau-1)}), \quad \tau = 1, 2, 3, \dots, \quad (24)$$

donde τ representa la iteración y η_τ el tamaño de paso de la iteración τ .

Después de cada iteración, se vuelve a calcular el gradiente para el nuevo vector de parámetros $\mathbf{w}^{(\tau)}$ y se repite el proceso, hasta que no se aprecien diferencias significativas entre una iteración y la siguiente o hasta alcanzar un número máximo de iteraciones fijado a priori. Criterios de parada incluyen:

$$\|\nabla R(\mathbf{w}^{(\tau)})\| < \epsilon \quad ; \quad \|\mathbf{w}^{(\tau)} - \mathbf{w}^{(\tau-1)}\| < \tilde{\epsilon} \quad ; \quad \|\nabla R(\mathbf{w}^{(\tau)}) - \nabla R(\mathbf{w}^{(\tau-1)})\| < \tilde{\tilde{\epsilon}} \quad (25)$$

para ciertas cotas ϵ , $\tilde{\epsilon}$, $\tilde{\tilde{\epsilon}}$.

Dos elementos determinantes en la implementación del algoritmo son la definición del valor inicial $\mathbf{w}^{(0)}$ y la elección de los tamaños de paso η_τ .

Con respecto a la elección del valor inicial, es importante ya que puede afectar tanto el tiempo de ejecución del algoritmo como la capacidad de generalización de la función de predicción asociada. Desafortunadamente, no existen, de momento, resultados teóricos que puedan guiar la elección de $\mathbf{w}^{(0)}$. Se aconseja elegir $\mathbf{w}^{(0)}$ de forma aleatoria y ejecutar el algoritmo para distintos valores iniciales $\mathbf{w}^{(0)}$ eligiendo el valor que produce la estimación del vector de parámetros que minimiza el valor de la función objetivo $R(\mathbf{w})$.

Con respecto a los tamaños de paso, la elección más sencilla es elegir un tamaño de paso constante, $\eta_\tau = \eta \forall \tau$. Sin embargo, dicha solución suele resultar bastante ineficiente. Para η demasiado grande, el algoritmo puede diverger, mientras que un valor de η demasiado pequeño podría resultar en un algoritmo muy lento e ineficiente. Intuitivamente, sería

conveniente utilizar métodos en los cuales el tamaño de paso η_τ es más grande en las primeras iteraciones y va disminuyendo conforme aumenta τ . Algunos ejemplos incluyen

$$\eta^{(\tau)} = (1 - \tau/Q)\eta^{(0)} + (\tau/Q)\eta^{(Q)} \quad (26)$$

$$\eta^{(\tau)} = \eta^{(0)}(1 + \tau/s)^{-\alpha} \quad (27)$$

donde en (26) el valor de η disminuye de forma lineal a lo largo de Q pasos, después de lo cual su valor se mantiene constante en $\eta^{(Q)}$. Los valores de los hiperparámetros $\eta^{(0)}$, $\eta^{(Q)}$, Q , s y α se determinan de forma empírica. En la práctica es recomendable monitorizar la curva de aprendizaje del error de entrenamiento durante la iteración del descenso del gradiente, para observar si disminuye a un ritmo adecuado.

También existen métodos más sofisticados como el algoritmo AdaGrad [13], RMSProp [14] o Adam [15], en los cuales el tamaño de paso es posiblemente distinto y disminuye de forma adaptativa para cada uno de los parámetros del vector $\mathbf{w}$.

Por otro lado, es importante observar que para evaluar el gradiente $\nabla R(\mathbf{w})$ en cada paso del algoritmo del descenso del gradiente, es necesario procesar el conjunto de datos de entrenamiento en su totalidad, lo que produce un coste computacional demasiado elevado para la mayoría de las aplicaciones (que utilizan conjuntos de entrenamiento muy grandes). En esos casos, una alternativa, que ha demostrado funcionar bien en muchas aplicaciones, es el método del descenso estocástico del gradiente (SGD) que se describe a continuación.

2.3.2. Descenso estocástico del gradiente

Para reducir el coste computacional del descenso del gradiente, es necesario realizar una aproximación del gradiente $\nabla R(\mathbf{w})$, sin tener que utilizar todo el conjunto de datos en cada iteración.

Teniendo en cuenta que en muchas aplicaciones el error de entrenamiento, $R(\mathbf{w})$, es la suma de N_{train} términos $R_n(\mathbf{w})$, cada uno relativo a una observación del conjunto de entrenamiento,

$$R(\mathbf{w}) = \sum_{n=1}^{N_{train}} R_n(\mathbf{w}) \quad (28)$$

la idea es que en cada iteración τ , el vector de parámetros $\mathbf{w}$ se actualice sobre la base de una única observación del conjunto de entrenamiento,

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta_\tau \nabla R_n(\mathbf{w}^{(\tau-1)}), \quad \tau = 1, 2, 3, \dots, \quad (29)$$

donde $\nabla R_n(\mathbf{w})$ es el gradiente del término $R_n(\mathbf{w})$ del error de entrenamiento correspondiente a la observación n -ésima, $n \in \{1, \dots, N_{train}\}$.

La actualización en (29) se repite de forma cíclica por todas las observaciones en el conjunto de entrenamiento.

Como se puede observar en el proceso iterativo (29), el conjunto de datos no se procesa en su totalidad, sino que, en cada iteración, se aproxima el gradiente total del error de entrenamiento únicamente con el gradiente del error de predicción correspondiente a una observación específica.

Las N_{train} iteraciones necesarias para procesar todo el conjunto de entrenamiento reciben el nombre de **época de entrenamiento**.

Otra propiedad del descenso del gradiente estocástico es la posibilidad de escapar de los mínimos locales, ya que un punto estacionario con respecto al error de entrenamiento para todo el conjunto de datos generalmente no será un punto estacionario para el error de predicción de una observación.

2.3.3. Descenso del gradiente por mini-lotes (mini-batch SGD)

Como hemos observado en las Secciones 2.3.1 y 2.3.2, el método del gradiente utiliza la totalidad del conjunto de datos de entrenamiento de tamaño N_{train} para calcular el gradiente. Este proceso tiene un elevado coste computacional cuando el conjunto de datos es grande. Por otro lado, el método estocástico del gradiente aproxima el gradiente utilizando una única observación del conjunto de datos de entrenamiento. Esto supone una gran ventaja computacional, sin embargo, presenta también una importante limitación, ya que la estimación del gradiente del error de entrenamiento basada en una única observación ($\nabla R_n(\mathbf{w})$), en general proporciona una estimación muy pobre del gradiente del error de entrenamiento que utiliza el conjunto de entrenamiento en su totalidad ($\nabla R(\mathbf{w})$).

El método del descenso del gradiente estocástico por mini-lotes proporciona una solución intermedia entre las dos anteriores. La idea es aproximar, en cada iteración τ , el gradiente del error de entrenamiento utilizando un subconjunto $\mathcal{B}_\tau$ de tamaño $B < N_{train}$, del conjunto de datos de entrenamiento. En particular, la aproximación del error de entrenamiento basada en $\mathcal{B}_\tau$, que denotaremos por $R_{\mathcal{B}_\tau}(\mathbf{w})$, se define como la media aritmética de los términos $R_n(\mathbf{w})$ del error de entrenamiento, correspondientes a las B observaciones en $\mathcal{B}_\tau$,

$$R_{\mathcal{B}_\tau}(\mathbf{w}) = \frac{1}{B} \sum_{n \in \mathcal{B}_\tau} R_n(\mathbf{w}) \quad (30)$$

Por tanto, el vector de parámetros $\mathbf{w}$ se actualiza de la forma siguiente,

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta_\tau \nabla R_{\mathcal{B}_\tau}(\mathbf{w}^{(\tau-1)}), \quad \tau = 1, 2, \dots, \quad (31)$$

donde $\nabla R_{\mathcal{B}_\tau}(\mathbf{w})$ es la aproximación del gradiente total obtenido utilizando el mini-lote $\mathcal{B}_\tau$. Si N_{train} es un múltiplo de B , cada época de entrenamiento contiene $\frac{N_{train}}{B}$ iteraciones.

Una observación importante es que, al utilizar los mini-lotes, los datos que los constituyen deben elegirse al azar del conjunto de datos de entrenamiento para que no haya correlación entre ellos. Como consecuencia, se suele realizar una permutación del conjunto de datos de entrenamiento para, posteriormente, extraer los mini-lotes como bloques consecutivos de tamaño B . Al principio de cada época de entrenamiento, el conjunto de datos se puede volver a permutar, para que sea improbable que se repitan los mini-lotes, lo que puede ayudar a escapar de mínimos locales.

2.3.4. Retropropagación

Como hemos observado en las secciones anteriores, los métodos basados en el descenso del gradiente requieren evaluar en cada iteración el gradiente del error de entrenamiento $R_n(\mathbf{w})$ relativas a un subconjunto $\tilde{\mathcal{B}}_\tau$ de las observaciones del conjunto de entrenamiento. En particular, en el método del descenso del gradiente, $\tilde{\mathcal{B}}_\tau = \{1, 2, \dots, N_{train}\}$ para todo τ , mientras que $\tilde{\mathcal{B}}_\tau$ contiene solo una observación y cíclicamente abarca todo el conjunto $\{1, \dots, N_{train}\}$ para el descenso del gradiente estocástico y $\tilde{\mathcal{B}}_\tau = \mathcal{B}_\tau$ para el descenso del gradiente estocástico con mini-lotes.

El cálculo directo de las derivadas parciales que definen $\nabla R_n(\mathbf{w})$ resulta computacionalmente muy costoso. Una alternativa eficiente para el cálculo de $\nabla R_n(\mathbf{w})$ es el método de **retropropagación**, que consta de tres fases:

1. La primera fase recibe el nombre de **propagación hacia delante** (*forward propagation*). En esta fase, la red recibe en la capa de entrada una observación del conjunto de entrenamiento. Posteriormente, la información se propaga hacia delante, es decir, fluye capa por capa calculando para cada neurona, primero su preactivación y después la activación de esta. Este proceso continúa hasta llegar a la capa de salida, donde se obtiene la predicción de la red.

2. Una vez obtenida la predicción de la red, se evalúa el término del error de entrenamiento asociado a dicha observación, $R_n(\mathbf{w})$, y se calcula la derivada de R_n con respecto a la preactivación de cada una de las unidades de la capa de salida.
3. Por último, la tercera fase recibe el nombre de **propagación hacia atrás** (*backward propagation*). En esta etapa, la información del error se propaga hacia atrás, desde la capa de salida hacia las capas anteriores. Como las neuronas ocultas no tienen un error propio observable que pueda comparar su activación con un valor real, se define el **término de error** como la influencia que tiene cada neurona de la red en el error final cometido, permitiendo posteriormente calcular $\nabla R_n(\mathbf{w})$.

El objetivo es calcular el ∇R_n , es decir, $\frac{\partial R_n}{\partial w_{ji}}$ para cada w_{ji} en el vector de parámetros. Para ilustrar el funcionamiento del método de retropropagación, vamos a considerar una red neuronal con una única capa oculta, donde el *input* es un vector de dimensión p , la capa oculta contiene K unidades y el *output* contiene C clases, como se muestra en la Figura 4.

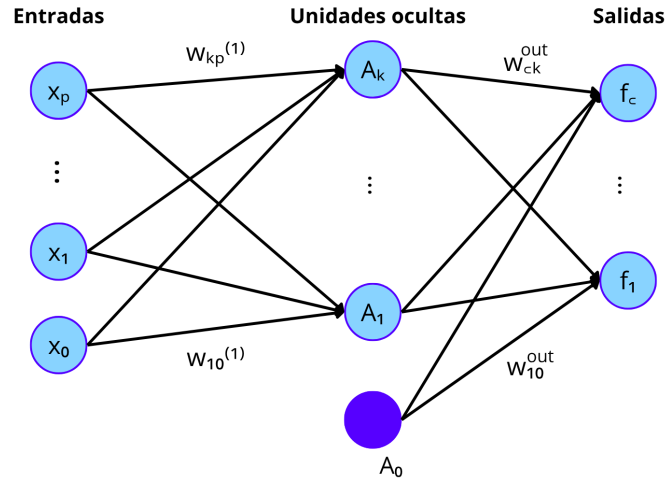


Figura 4: Ilustración de una red con una única capa oculta. Las flechas indican la dirección del flujo de información en la red durante la propagación hacia delante.

Como se muestra en la Figura 4, para simplificar la notación se ha introducido una unidad adicional en cada capa de la red. En particular, se introduce $x_{n0} = 1$ en la capa de entrada y $A_0^{(l)} = 1$, $l \in \{1, \dots, L\}$ en las capas posteriores para que el sesgo quede incluido dentro de los pesos, por lo que $w_{j0} \cdot A_0 = w_{j0} \cdot 1 = w_{j0}$.

Para la n -ésima observación de los datos de entrenamiento, sean:

- $\mathbf{X}_n = (x_{n1}, x_{n2}, \dots, x_{np}) \in \mathbb{R}^p$ el vector de entrada
- $Y_n = (Y_{n1}, Y_{n2}, \dots, Y_{nC}) \in \mathbb{R}^C$ el vector de salida en la codificación 1-de- K , es decir,

$$Y_{nc} = \begin{cases} 1 & \text{si la clase verdadera del } n\text{-ésimo individuo es la clase "c"} \\ 0 & \text{de lo contrario} \end{cases} \quad (32)$$

La función de activación que se utilizará en la capa de salida es la *softmax*,

$$f_c(\mathbf{X}_n, \mathbf{w}) = \frac{e^{a_c^{\text{out}}(\mathbf{X}_n, \mathbf{w})}}{\sum_{l=1}^C e^{a_l^{\text{out}}(\mathbf{X}_n, \mathbf{w})}} \quad (33)$$

donde $a_c^{\text{out}}(\mathbf{X}_n, \mathbf{w})$ es la preactivación de la c -ésima unidad de salida.

Por otro lado, utilizamos como función de activación de las unidades ocultas la función $\tanh(\cdot)$,

$$g(a) = \tanh(a) \quad . \quad (34)$$

Finalmente, la función de error utilizada será la entropía cruzada,

$$R_n(\mathbf{w}) = - \sum_{c=1}^C Y_{nc} \cdot \ln(f_c(\mathbf{X}_n, \mathbf{w})) \quad . \quad (35)$$

El algoritmo de retropropagación, en este caso, consistiría de los siguientes pasos:

▪ **Paso 1 (propagación hacia adelante)**

El primer paso es propagar hacia delante $\mathbf{X}_n$. Abreviando $f_c(\mathbf{X}_n, \mathbf{w})$ y $a_c^{\text{out}}(\mathbf{X}_n, \mathbf{w})$ por f_{cn} y a_c^{out} respectivamente, se calculan las preactivaciones y activaciones de las unidades ocultas

$$a_j^{(1)} = \sum_{i=0}^p w_{ji}^{(1)} \cdot x_{ni} \quad , \quad j = 1, \dots, K, \quad (36)$$

$$A_j^{(1)} = g(a_j^{(1)}) \quad , \quad j = 1, \dots, K, \quad (37)$$

y las preactivaciones y activaciones de las unidades de salida

$$a_c^{\text{out}} = \sum_{h=0}^K w_{ch}^{\text{out}} \cdot A_h^{(1)} \quad , \quad c = 1, \dots, C, \quad (38)$$

$$f_{cn} = \frac{e^{a_c^{\text{out}}}}{\sum_{l=1}^C e^{a_l^{\text{out}}}} \quad , \quad c = 1, \dots, C \quad . \quad (39)$$

■ **Paso 2 (cálculo de R_n y de $\delta_c^{\text{out}} = \frac{\partial R_n}{\partial a_c^{\text{out}}}$)**

A continuación, calculamos $\delta_c^{\text{out}} = \frac{\partial R_n}{\partial a_c^{\text{out}}}$ para las unidades de salida ($c = 1, \dots, C$).

Sabemos que

$$R_n(\mathbf{w}) = - \sum_{l=1}^C Y_{nl} \cdot \ln \left(\frac{e^{a_l^{\text{out}}}}{\sum_{r=1}^C e^{a_r^{\text{out}}}} \right) = - \sum_{l=1}^C Y_{nl} \cdot [a_l^{\text{out}} - \ln \left[\left(\sum_{r=1}^C e^{a_r^{\text{out}}} \right) \right]] \quad . \quad (40)$$

Por otro lado, tenemos:

$$\begin{aligned} \frac{\partial}{\partial a_c^{\text{out}}} \left[a_l^{\text{out}} - \ln \left(\sum_{r=1}^C e^{a_r^{\text{out}}} \right) \right] &= \begin{cases} 0 - \frac{1}{\sum_{r=1}^C e^{a_r^{\text{out}}}} \cdot e^{a_c^{\text{out}}} & \text{si } l \neq c \\ 1 - \frac{1}{\sum_{r=1}^C e^{a_r^{\text{out}}}} \cdot e^{a_c^{\text{out}}} & \text{si } l = c \end{cases} \\ &= \begin{cases} -f_{cn} & \text{si } l \neq c \\ 1 - f_{cn} & \text{si } l = c \end{cases} \end{aligned} \quad (41)$$

Por tanto,

$$\frac{\partial R_n(\mathbf{w})}{\partial a_c^{\text{out}}} = - \sum_{l \neq c} Y_{nl} \cdot (-f_{cn}) - Y_{nc} \cdot (1 - f_{cn}) = f_{cn} \sum_{l=1}^C Y_{nl} - Y_{nc} = f_{cn} - Y_{nc} \quad (42)$$

Como consecuencia, obtenemos que la derivada parcial de R_n con respecto a la preactivación de la unidad c de la capa de salida se puede expresar como la diferencia entre la probabilidad predicha de que el n -ésimo individuo pertenezca a la clase c y la probabilidad real de que el individuo pertenezca a la clase c (esta última será 0 o 1),

$$\delta_c^{\text{out}} = f_{cn} - Y_{nc}, \quad c = 1, \dots, C \quad . \quad (43)$$

■ **Paso 3 (retropropagación)**

Retropropagamos δ_c^{out} para obtener los $\delta_j^{(1)} = \frac{\partial R_n}{\partial a_j^{(1)}}$ de las unidades ocultas. Para ello utilizamos la regla de la cadena

$$\delta_j^{(1)} = \frac{\partial R_n}{\partial a_j^{(1)}} = \sum_{c=1}^C \frac{\partial R_n}{\partial a_c^{out}} \cdot \frac{\partial a_c^{out}}{\partial a_j^{(1)}} \quad . \quad (44)$$

Por otro lado, nuestro objetivo es calcular la derivada parcial $\frac{\partial a_c^{out}}{\partial a_j^{(1)}}$. Por (38) se tiene

$$a_c^{out} = \sum_{h=0}^K w_{ch}^{out} \cdot A_h^{(1)} \quad , \quad c = 1, \dots, C, \quad (45)$$

y como consecuencia, podemos calcular la derivada parcial de a_c^{out} con respecto a $a_j^{(1)}$.

$$\frac{\partial a_c^{out}}{\partial a_j^{(1)}} = \frac{\partial}{\partial a_j^{(1)}} \left(\sum_{h=0}^K w_{ch}^{out} \cdot A_h^{(1)} \right) = \frac{\partial}{\partial a_j^{(1)}} \left(\sum_{h=1}^K w_{ch}^{out} \cdot g(a_h^{(1)}) \right) \quad , \quad c = 1, \dots, C \quad . \quad (46)$$

Como $a_c^{out} = w_{c0}^{out} + w_{c1}^{out} \cdot g(a_1^{(1)}) + w_{c2}^{out} \cdot g(a_2^{(1)}) + \dots + w_{cj}^{out} \cdot g(a_j^{(1)}) + \dots + w_{cK}^{out} \cdot g(a_K^{(1)})$, se obtiene:

$$\frac{\partial a_c^{out}}{\partial a_j^{(1)}} = \frac{\partial}{\partial a_j^{(1)}} \left(w_{cj}^{out} \cdot g(a_j^{(1)}) \right) = w_{cj}^{out} \cdot g'(a_j^{(1)}) \quad (47)$$

Sustituyendo la expresión (47) en la expresión (44), se obtiene la *fórmula de retro-propagación*:

$$\begin{aligned} \delta_j^{(1)} &= \frac{\partial R_n}{\partial a_j^{(1)}} = \sum_{c=1}^C \delta_c^{out} \cdot \frac{\partial a_c^{out}}{\partial a_j^{(1)}} = \sum_{c=1}^C \delta_c^{out} \cdot \left(w_{cj}^{out} \cdot g'(a_j^{(1)}) \right) = \\ &= g'(a_j^{(1)}) \cdot \sum_{c=1}^C w_{cj}^{out} \cdot \delta_c^{out} = \left[1 - (A_j^{(1)})^2 \right] \cdot \sum_{c=1}^C w_{cj}^{out} \cdot \delta_c^{out}, \quad j = 1, \dots, K \end{aligned} \quad (48)$$

- Como resultado de los pasos anteriores, las derivadas parciales de R_n con respecto a los parámetros de la capa oculta y la capa de salida del modelo (que definen el gradiente de $R_n(\mathbf{w})$), aplicando la regla de la cadena, se pueden expresar como:

$$\begin{aligned} \frac{\partial R_n}{\partial w_{ji}^{(1)}} &= \frac{\partial R_n}{\partial a_j^{(1)}} \cdot \frac{\partial a_j^{(1)}}{\partial w_{ji}^{(1)}} = \delta_j^{(1)} \cdot x_{ni} \quad , \quad j = 1, \dots, K, \quad i = 0, \dots, p, \\ \frac{\partial R_n}{\partial w_{cj}^{out}} &= \frac{\partial R_n}{\partial a_c^{out}} \cdot \frac{\partial a_c^{out}}{\partial w_{cj}^{out}} = \delta_c^{out} \cdot A_j^{(1)} \quad , \quad c = 1, \dots, C, \quad j = 0, \dots, K \quad . \end{aligned} \quad (49)$$

2.4. Ejemplo práctico (MNIST)

Para ilustrar la implementación de las redes neuronales prealimentadas presentadas en las secciones anteriores, utilizaremos el conjunto de datos **MNIST** [7] (del nombre en inglés *Modified National Institute of Standards and Technology*) que contiene 70.000 imágenes en escala de grises de tamaño 28×28 píxeles. Cada imagen representa un número manuscrito del 0 al 9. Una muestra de 40 imágenes del conjunto MNIST se muestra en la Figura 5.

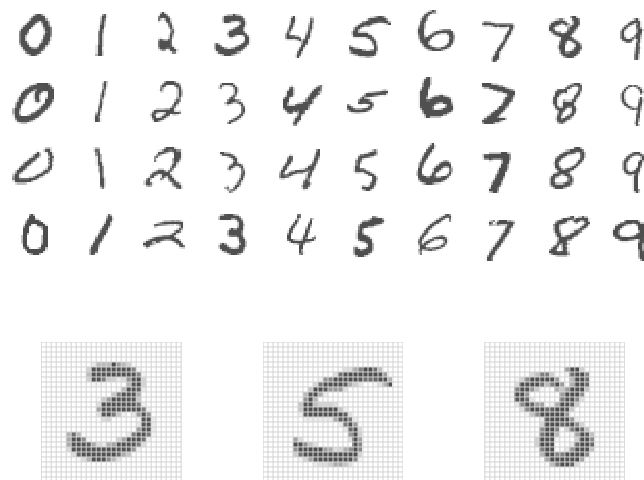


Figura 5: Muestra de 40 imágenes del conjunto MNIST. Los primeros 3, 5 y 8 se muestran ampliados para observar sus $28 \times 28 = 784$ valores de píxeles individuales.

Fuente: An Introduction to Statistical Learning: with Applications in Python [2] Figura 10.3, Página 403.

Las 70.000 imágenes están divididas en un conjunto de entrenamiento, de tamaño 60.000, y un conjunto test de tamaño 10.000. Para cada imagen, el *input* se representa como un vector $\mathbf{X}$ de longitud $28 \times 28 = 784$. Cada elemento del vector representa la intensidad del píxel correspondiente (un valor entero entre 0 y 255).

Por otro lado, el *output* de cada imagen es el número representado en la misma. El objetivo es definir y entrenar una red neuronal prealimentada que permita predecir (clasificar) correctamente el número representado en una nueva imagen, es decir, que asocie a cada vector del *input*, $\mathbf{X} = (X_1, \dots, X_{784})$, que contiene las intensidades de los píxeles de una imagen, una predicción del *output* (un número del 0 al 9) minimizando el error de predicción. Por lo tanto, se trata de un problema de clasificación multiclase con $C = 10$ clases.

Como primera aproximación al problema, consideremos una red neuronal con una única capa intermedia con 256 neuronas. Para las unidades de la capa intermedia, utilizaremos la función de activación *ReLU*. Para las $C = 10$ unidades de la capa de salida, utilizaremos la función de activación *softmax*. La arquitectura de la red se muestra en la Figura 6.

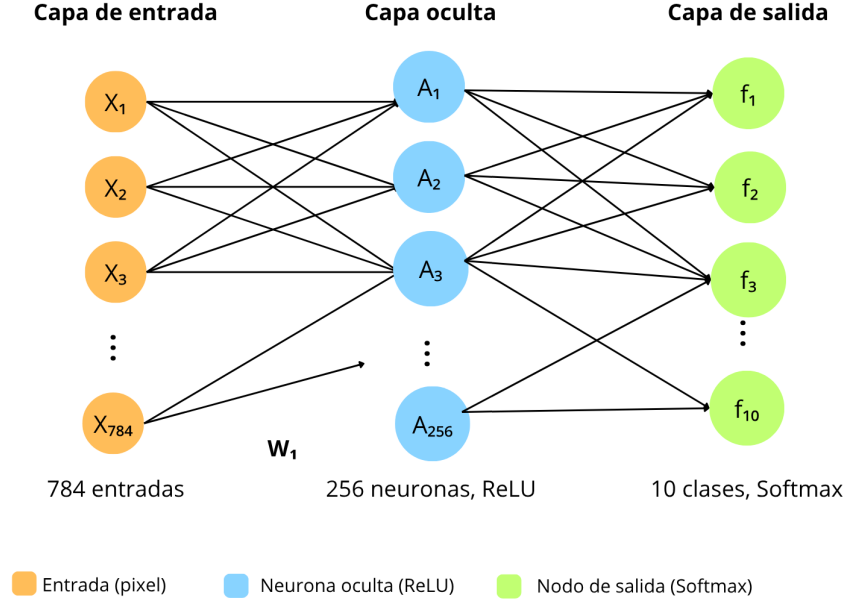


Figura 6: Esquema de la red neuronal prealimentada utilizada en el ejemplo MNIST: 784 neuronas en la capa de entrada, una capa oculta de 256 neuronas y 10 neuronas en la capa de salida.

Obsérvese que, al utilizar la función *softmax*, la activación de la neurona h de la capa de salida, $f_h(\mathbf{X}, \mathbf{w})$ se puede interpretar como la probabilidad, predicha por la FNN, de que la imagen corresponda al número h . Por lo tanto, como el objetivo es construir un clasificador, la FNN, en realidad, estima la probabilidad de que el número representado en una imagen sea h para $h \in \{0, 1, \dots, 9\}$. El clasificador basado en la FNN, asignará la imagen a la clase h^* con la probabilidad más alta de ser la clase correcta.

El número total de parámetros de la FNN, en este caso es $d = (784 + 1) \cdot (256) + (256 + 1) \cdot 10 = 203.530$.

Para el entrenamiento de la red, se ha utilizado el 80 % (es decir, 48.000 imágenes, del conjunto de entrenamiento), reservando el restante 20 % (es decir, 12.000 imágenes, para la validación). Para el entrenamiento se ha utilizado la entropía cruzada como función de error, mini-lotes de tamaño 256, 30 épocas y el algoritmo de optimización RMS-Prop [14], implementado en Python utilizando los paquetes ISLP [2], PyTorch [12] y

`pytorch_lightning` [16].

Para cuantificar el rendimiento de la red neuronal en el conjunto test y monitorizar el aprendizaje de la FNN en el proceso de entrenamiento, se ha utilizado la tasa de acierto que mide el porcentaje de clasificaciones (predicciones) correctas,

$$\text{Tasa de acierto} = \frac{\text{número de predicciones correctas}}{\text{número total de observaciones}} \times 100 \quad (50)$$

El uso de la tasa de acierto puede ser adecuado en ese caso, teniendo en cuenta que el conjunto MNIST está aproximadamente balanceado, es decir, contiene una cantidad similar de imágenes de cada uno de los 10 dígitos.

La tasa de acierto de la red en el conjunto test es del 97,29 %, bastante satisfactoria. Con respecto al proceso de entrenamiento, en la Figura 7 se muestra la evolución de la tasa de acierto en el conjunto de entrenamiento (de 48.000 imágenes) y en el conjunto de validación, como función de la época de entrenamiento.

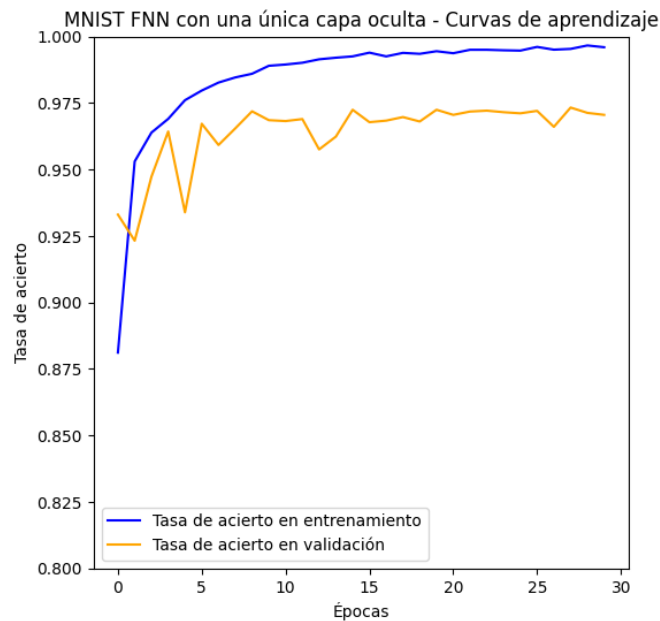


Figura 7: Curvas de aprendizaje del modelo descrito en la Figura 6.

Podemos observar cómo durante las primeras épocas, las tasas de acierto en entrenamiento y validación aumentan rápidamente, lo que indica que el modelo es capaz de ajustar con facilidad los patrones del conjunto MNIST. Sin embargo, en torno a la época 8, la curva de entrenamiento sigue aumentando hasta aproximarse a 1, mientras que la curva de validación se mantiene prácticamente estable. La separación entre ambas curvas

de aprendizaje indica un sobreajuste en el modelo, es decir, la red continúa mejorando la adaptabilidad a los datos de entrenamiento, mientras que su capacidad de generalización sobre datos no vistos no mejora.

Por último, a partir de la época 15, apenas mejora la tasa de acierto en validación. Es por ello que, en este momento, sería razonable detener el entrenamiento de la red, ya que la mejoría solamente afecta a la curva de entrenamiento.

Las principales características de la FNN con una única capa intermedia y su rendimiento en el conjunto test se resumen en la Tabla 2.

FNN DE UNA ÚNICA CAPA	
Arquitectura de Capas	784 $\rightarrow$ 256 $\rightarrow$ 10
Parámetros Totales	203.530
Tasa de Acierto	97,29 %

Tabla 2: Resultados del modelo descrito en la Figura 6.

Al aumentar el número de capas ocultas no hemos conseguido mejorar los resultados de la FNN descrita en la Tabla 2.

3. Redes Neuronales Convolucionales (CNN)

Los modelos de aprendizaje más simples asumen que no se dispone de ningún tipo de información sobre la estructura de los datos de entrada, es decir, sobre las posibles relaciones entre los elementos del vector de entrada $\mathbf{X} = (X_1, \dots, X_p)$.

Como consecuencia, si se realizan permutaciones entre los elementos del vector $\mathbf{X}$, el rendimiento del modelo no cambiará.

Sin embargo, en muchas aplicaciones de interés, los datos de entrada presentan una estructura bien definida y conocida que proporciona información valiosa sobre las relaciones entre los elementos del vector de entrada. Por ejemplo, en problemas de clasificación de imágenes, los elementos del vector de entrada representan las intensidades de los píxeles de la imagen que tienen una relación espacial bien definida y conocida (para una imagen de dimensión $J \times H$, el vector de entrada contiene $J \cdot H$ elementos) y píxeles cercanos tienden a tener intensidades correlacionadas.

En estos casos, es posible utilizar la información de la que se dispone sobre la estructura de los datos de entrada, para diseñar la arquitectura del modelo de red neuronal de forma que el modelo, por construcción, tenga ciertas propiedades relevantes para el problema de interés. Por ejemplo, en un problema de clasificación de imágenes, el modelo deberá ser capaz de identificar la presencia de un tigre en una imagen independientemente de la posición del tigre en la imagen (invarianza respecto a traslaciones) y determinar la posición del tigre en la imagen (equivarianza respecto a traslaciones). El objetivo, por lo tanto, es utilizar la información de la que se dispone sobre la imagen, para diseñar un modelo de red neuronal que, por construcción, posea estas propiedades. Esta es exactamente la idea de las redes convolucionales (CNN, del nombre en inglés *Convolutional Neural Network*) que fueron introducidas por Yann LeCun y su equipo en 1989 [10] para el procesamiento de imágenes (concretamente el reconocimiento de códigos postales escritos a mano).

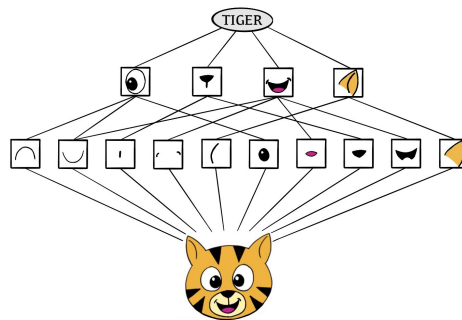


Figura 8: Ejemplo del proceso de clasificación de una imagen en una CNN. Se puede observar cómo la red identifica las características de bajo nivel del tigre, como son bordes, contornos y texturas, mientras que las capas siguientes las combinan para obtener características compuestas y lograr la clasificación exitosa de la imagen.

Fuente: An Introduction to Statistical Learning: with Applications in Python [2], Figura 10.6, Página

Además de la invarianza y equivarianza respecto a traslaciones, un tercer concepto, de fundamental importancia en la definición de la arquitectura de las redes convolucionales, es el concepto de jerarquía. Supongamos, por ejemplo, que el objetivo es detectar caras de un tigre en imágenes (véase Figura 8). En ese caso existe una estructura jerárquica natural ya que la imagen puede contener varias caras y cada cara está compuesta por varios elementos como los ojos, la boca, las orejas (con una relación espacial bien definida entre ellos) y que a la vez presentan una estructura bien definida formada por elementos más pequeños (el ojo, por ejemplo, está formado por el iris y la pupila) que a la vez pueden descomponerse en elementos más pequeños (como bordes y líneas). Al nivel más bajo de la jerarquía,

una unidad (neurona) de la red convolucional podría detectar la presencia de uno de los elementos más pequeños de la cara (como bordes o líneas) utilizando la información local relativa a los píxeles de una región pequeña de la imagen. Estructuras más complejas, correspondientes a niveles más altos de la jerarquía que define la estructura de la cara del tigre, pueden ser detectadas combinando múltiples estructuras sencillas detectadas en los niveles más bajos (véase Figura 8). Las distintas capas intermedias de una red convolucional corresponden a los distintos niveles de este aprendizaje jerárquico.

En las primeras capas, la red aprende cuáles son las estructuras de bajo nivel (los elementos más sencillos de la cara de un tigre) y cómo identificarlos. En las capas sucesivas, la red aprende a combinar las estructuras de las capas anteriores para definir e identificar estructuras más complejas, hasta la capa de salida donde finalmente la red predice con qué probabilidad la imagen en cuestión contiene una (o varias) cara de un tigre.

La arquitectura de una red convolucional se basa en la combinación sucesiva de dos tipos de capas ocultas:

- **Capas de Convolución:** Su responsabilidad es la búsqueda, mediante *filtros de convolución*, de pequeños patrones de la imagen.
- **Capas de Agrupación (Pooling):** Su función es la de realizar un submuestreo a los patrones encontrados anteriormente para reducir la dimensionalidad y aportar cierta invarianza a pequeñas traslaciones.

A continuación, explicaremos la arquitectura y las características de cada una de estas capas.

3.1. Detectores de características

Para ilustrar cómo se realiza la detección de características, consideramos el caso más sencillo, en el cual la imagen de entrada, I , de dimensión $J \times H$, tiene un único canal (es decir, es una imagen en escala de grises), y denotamos por $I(j, h)$ la intensidad del píxel (j, h) , donde $j = 1, \dots, J$, $h = 1, \dots, H$.

Una unidad (neurona) de la primera capa oculta, denominada **capa convolucional**, toma como entrada las intensidades de los píxeles de una región pequeña de la imagen I . Dicha región recibe el nombre de **campo receptivo** de la unidad y formaliza la noción de localidad (véase Figura 9). Supondremos, por el momento, que el campo receptivo tiene dimensión $M \times M$, con $M < \min\{J, H\}$ (aunque los campos receptivos rectangulares son posibles, no son muy comunes en las aplicaciones).

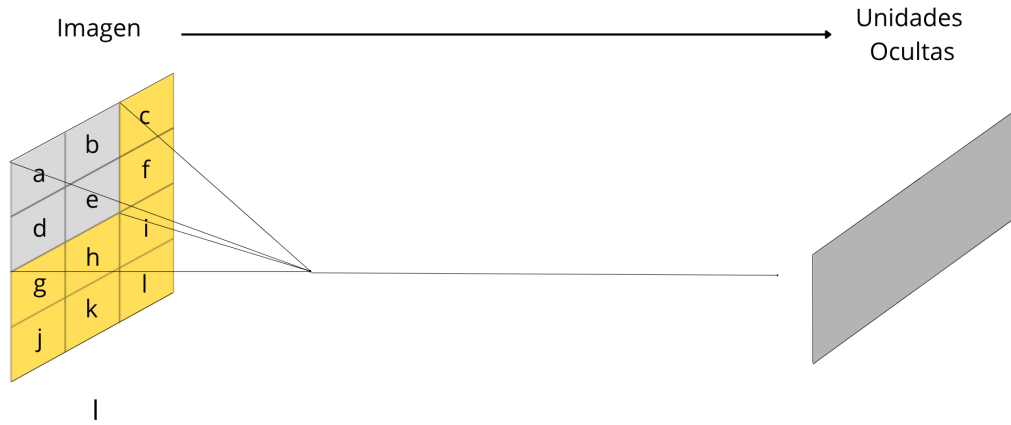


Figura 9: Ilustración del campo receptivo de una unidad de la primera capa oculta sobre la imagen de entrada I .

A cada píxel del campo receptivo de la unidad va asociado un peso, y estos pesos, en conjunto, definen un **filtro** (o *kernel*) de la misma dimensión del campo receptivo ($M \times M$) y que denotaremos por K .

La preactivación y la salida de la unidad vienen dadas por

$$a = \mathbf{w}^T \mathbf{x} + w_0 \quad (51)$$

$$A = g(\mathbf{w}^T \mathbf{x} + w_0) \quad (52)$$

donde $\mathbf{x}$ es el vector formado por los valores de los píxeles del campo receptivo, $\mathbf{w}$ es el vector de pesos que definen el filtro K , w_0 es el sesgo y $g(\cdot)$ es la función de activación de la unidad (véase Figura 10).

Idealmente, los pesos del campo receptivo deberían permitir a la unidad capturar características locales de la imagen, de bajo nivel dentro de la jerarquía que define la imagen (bordes, líneas, etc.).

Para ello, es importante observar que de acuerdo con la ecuación (51), fijando el valor del vector de pesos $\mathbf{w}$ y del sesgo w_0 , la máxima preactivación de la unidad se alcanza (suponiendo que $\|\mathbf{x}\|$ es fija) cuando $\mathbf{x} = \alpha \mathbf{w}$, donde $\alpha > 0$. Esto quiere decir que la máxima preactivación se alcanza cuando la subimagen del campo receptivo ($\mathbf{x}$) es parecida a la "imagen" del filtro ($\mathbf{w}$) (en el sentido de que las intensidades de los píxeles del campo receptivo coinciden, a menos de una constante de proporcionalidad, con los pesos del filtro K de la unidad).

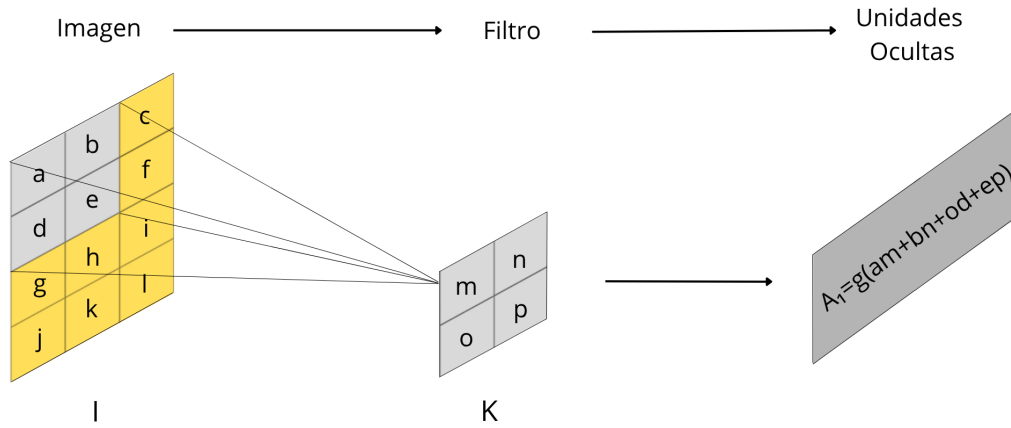


Figura 10: Representación de la aplicación de un filtro, K , de dimensión 2×2 sobre un campo receptivo de la imagen I .

El proceso de entrenamiento de la red, que determina los pesos y por lo tanto el filtro de la unidad, es el que permite la detección de características locales y de bajo nivel en la jerarquía de la imagen. De acuerdo con lo anterior, la señal emitida por la neurona (es decir, la salida de la unidad) será máxima cuando la subimagen asociada al campo receptivo ($\mathbf{x}$) sea parecida al filtro de la neurona (que representa la característica local, de bajo nivel, aprendida por la red).

3.2. Equivarianza a la traslación

Una CNN debe ser capaz de identificar la posición en la imagen de una cierta característica (bordes, líneas, ojos, orejas), es decir, detectar la presencia de la característica en una región de la imagen y reconocerla en otras posiciones sin necesidad de aprenderla nuevamente en cada ubicación.

Para ello, en una CNN, las unidades que forman cierta capa convolucional intermedia se obtienen desplazando el campo receptivo, de manera sistemática sobre la imagen, manteniendo fijos los pesos del correspondiente filtro. El número de unidades (neuronas) de la capa convolucional es igual al número de desplazamientos posibles del campo receptivo.

La capa convolucional puede visualizarse como una imagen (véase Figura 11) que recibe el nombre de **mapa de características**, y (tras una posible transformación intermedia realizada por la capa de *pooling*) representa la entrada de la siguiente capa convolucional (véase Figura 11).

La operación que produce el mapa de características, es decir, una capa convolucional,

se conoce como ”**convolución**”, y la denotamos por ” $*$ ”.

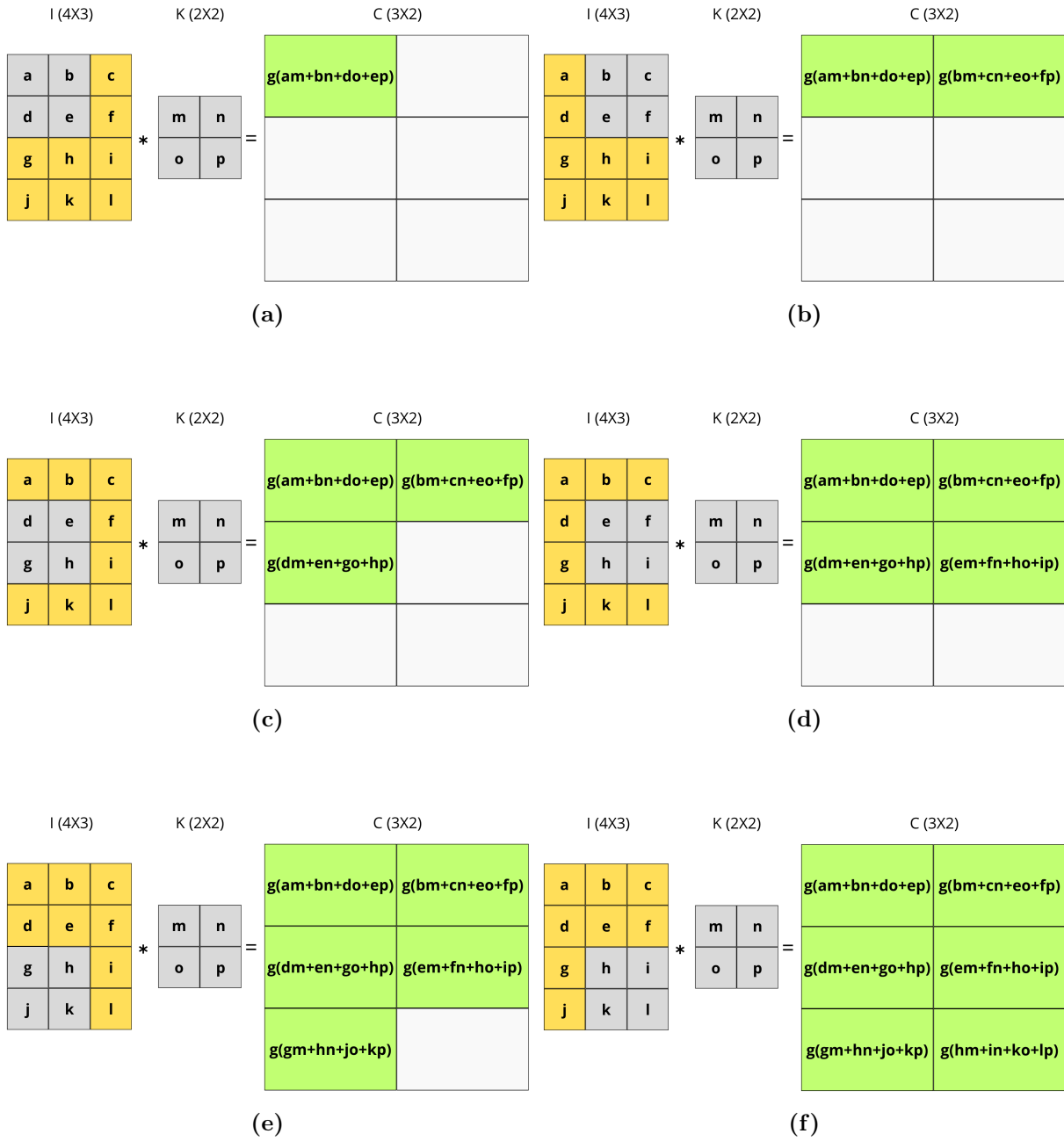


Figura 11: Ilustración del desplazamiento del campo receptivo sobre la imagen de entrada, I , de dimensión 4×3 , manteniendo el mismo filtro, K , de dimensión 2×2 y siendo $g(\cdot)$ la función de activación de la unidad. Como consecuencia, se obtiene como resultado el mapa de características C de dimensión 3×2 (operación de convolución): (a) Aplicación inicial del filtro, (b) Segundo desplazamiento, (c) Tercer desplazamiento, (d) Cuarto desplazamiento, (e) Quinto desplazamiento, (f) Último desplazamiento.

Por ejemplo, si la imagen de entrada de la CNN tiene dimensión $J \times H$ y el campo receptivo para la primera capa convolucional tiene dimensión $L \times M$, el mapa de características, C , tendrá dimensión $(J - L + 1) \times (H - M + 1)$ y se tiene (la notación utilizada fue introducida por Vincent Dumoulin y Francesco Visin en el año 2016 [8]):

$$C(j, k) = g \left(\sum_{l=1}^L \sum_{m=1}^M I(j + l - 1, k + m - 1) K(l, m) \right) \quad (53)$$

$$\text{con } j \in \{1, \dots, J - L + 1\}, \quad k \in \{1, \dots, H - M + 1\}$$

donde I es la imagen de entrada de dimensión $J \times H$, siendo $I(j, h)$ la intensidad del píxel (j, h) , K es el filtro de dimensión $L \times M$, siendo $K(l, m)$ el valor del peso (l, m) , $g(\cdot)$ es la función de activación de la unidad y $C(j, k)$ la componente (j, k) del mapa de características C .

La ecuación (53) se puede expresar matricialmente como:

$$C = I * K \quad (54)$$

Por otro lado, la primera capa convolucional tendrá $(J - L + 1) \cdot (H - M + 1)$ neuronas (véase Figura 12).

Como se puede observar en la Figura 12, la capa convolucional formaliza la noción de equivarianza respecto a la traslación ya que, si el campo receptivo se desplaza a otra ubicación en la imagen de entrada, las salidas asociadas al mapa de características se moverán a la ubicación correspondiente. Esto se produce gracias a las conexiones locales de las capas convolucionales y la compartición de pesos, ya que al aplicar el mismo filtro a distintas regiones de la imagen de entrada, la red puede detectar las mismas características independientemente de su ubicación.

3.3. Padding (relleno)

Como se ha observado en la sección anterior, para una imagen de entrada de dimensión $J \times H$, la utilización de un campo receptivo de dimensión $L \times M$ produce una primera capa convolucional (oculta) con $(J - L + 1) \cdot (H - M + 1)$ unidades y un mapa de características $(J - L + 1) \times (H - M + 1)$ que (tras una posible transformación de *pooling*) representa la imagen de entrada para la siguiente capa convolucional. La iteración de dichos procesos define la arquitectura de la red neuronal.

Sin embargo, dicha iteración tiene dos limitaciones:

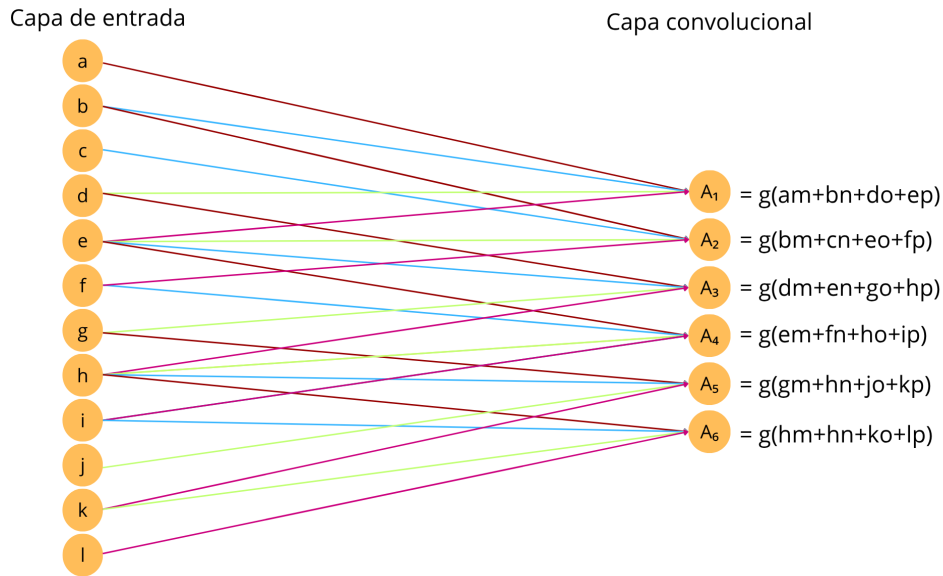


Figura 12: Representación de la compartición de pesos y las conexiones locales en una capa convolucional mediante el desplazamiento del campo receptivo, manteniendo fijos los pesos del filtro, K , sobre la imagen de entrada. Las flechas del mismo color representan conexiones asociadas al mismo peso, lo que permite detectar una misma característica en diferentes ubicaciones.

- En redes neuronales con múltiples capas ocultas, provocaría que el tamaño de la imagen disminuyese progresivamente en cada etapa, limitando el número de capas que puede tener una red.
- Al deslizar el campo receptivo por la imagen original, los píxeles del centro actúan en la operación más de una vez, mientras que los píxeles de los bordes solo son evaluados en una o pocas ocasiones, provocando que la red otorgue menor importancia a la información de la imagen en los bordes.

Para poder solucionar estos problemas se utiliza una técnica llamada **padding (relleno)**. El **padding** consiste en añadir un borde de ceros alrededor de la imagen de entrada antes de aplicar la convolución para así obtener un mapa de características de las mismas dimensiones que la imagen original.

Si añadimos P píxeles, el mapa de características resultante tendrá una dimensión de $(J + 2P - L + 1) \times (H + 2P - M + 1)$. Además, si el valor de P se elige de forma que el mapa de características tenga la misma dimensión que la imagen de entrada, la correspondiente convolución recibe el nombre de **convolución idéntica**.

Por ejemplo, si el mapa de características resultante tiene dimensión $(J + 2P - M + 1) \times (H + 2P - M + 1)$, por lo tanto, si la imagen de entrada tiene dimensión $J \times H$,

comparando expresiones obtenemos que el valor de P debe ser:

$$\begin{aligned} J + 2P - M + 1 &= J \rightarrow P = \frac{M - 1}{2} \\ H + 2P - M + 1 &= H \rightarrow P = \frac{M - 1}{2} \end{aligned} \quad (55)$$

En consecuencia, tomando $P = \frac{M - 1}{2}$ (asumiendo que M es impar), la imagen y el mapa de características tendrán las mismas dimensiones.

La Figura 13 muestra el efecto de aplicar *padding* con $P = 1$ sobre una imagen de entrada de dimensión 4×4 . Como resultado, se obtiene una matriz ampliada de dimensión 6×6 . Posteriormente, si se aplicase la convolución con un filtro, K , de dimensión 3×3 , se obtendría un mapa de características 4×4 , es decir, la misma dimensión que la imagen de entrada (este sería un caso de convolución idéntica).

0	0	0	0	0	0
0	x_{11}	x_{12}	x_{13}	x_{14}	0
0	x_{21}	x_{22}	x_{23}	x_{24}	0
0	x_{31}	x_{32}	x_{33}	x_{34}	0
0	x_{41}	x_{42}	x_{43}	x_{44}	0
0	0	0	0	0	0

Figura 13: Ilustración de una imagen de entrada 4×4 a la que se le ha aplicado un padding con $P = 1$, obteniendo como resultado una matriz ampliada de dimensión 6×6 .

Mediante la convolución idéntica logramos que la red preserve la dimensionalidad, asegurando que se reduzca la pérdida de información de los píxeles de los bordes.

3.4. Convoluciones con paso (Stride)

En el procesamiento de imágenes, para una imagen de entrada, I , de dimensión $J \times H$, el número de píxeles que la forman es $J \cdot H$. Además, en las aplicaciones se utilizan filtros, K , de dimensión $M \times M$ con $M < \min\{J, H\}$. Por tanto, el mapa de características, C , tendrá una dimensión próxima a la imagen de entrada, I , y será del mismo tamaño si se utiliza un *padding* adecuado. Sin embargo, no siempre conviene mantener mapas de características de gran tamaño en todas las capas convolucionales, lo cual provocaría

un gran coste computacional y una mayor complejidad de la red. Es por ello que, a veces, se necesita reducir la dimensión del mapa de características para disminuir el coste computacional y controlar las representaciones intermedias.

Como se ilustra en la Figura 14a, el filtro se desplaza sobre la imagen de entrada píxel a píxel, sin embargo, el desplazamiento puede modificarse. El número de píxeles que el filtro se desplaza en cada dirección recibe el nombre de **stride (paso)** y lo denotamos por S .

Como se observa en la Figura 14b, cuando $S > 1$ se realiza un submuestreo durante la convolución (conocido como **convolución con paso**), reduciendo el grado de solapamiento entre los campos receptivos y brindando una mayor flexibilidad en el diseño de la red, obteniendo así un mapa de características de menor dimensión.

Si aplicamos un filtro, K , de dimensión $M \times M$, con *padding* P , a una imagen de entrada, I , de dimensión $J \times H$ y un *stride* S , las dimensiones del mapa de características, C , resultante son:

$$\left(\left\lfloor \frac{J + 2P - M}{S} \right\rfloor + 1 \right) \times \left(\left\lfloor \frac{H + 2P - M}{S} \right\rfloor + 1 \right) \quad (56)$$

donde $\lfloor \cdot \rfloor$ es la función parte entera inferior. Se puede observar de la ecuación (56) que si $S > 1$, la dimensión del mapa de características se reduce.

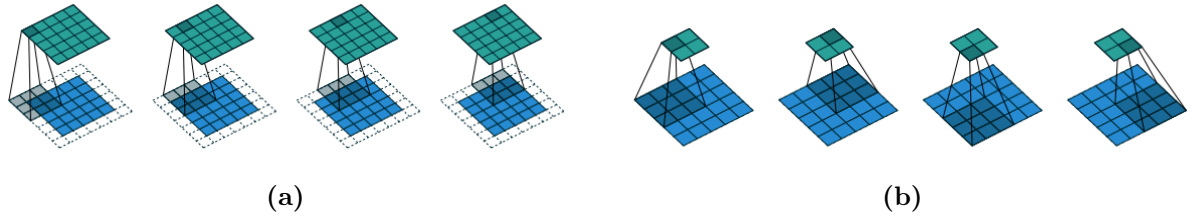


Figura 14: Ilustración de convoluciones con diferentes valores de stride: (a) Ejemplo de convolución con padding $P = 1$ y stride $S = 1$. (b) Ejemplo de convolución con stride $S = 2$.

Fuente: *A guide to convolution arithmetic for deep learning* [8], Figuras 2.3 y 2.5, Páginas 14 y 17.

3.5. Capas de agrupación (pooling layer)

La arquitectura jerárquica que presentan las redes neuronales, donde las características complejas se construyen a partir de características más simples, es sumamente importante, al igual que la relación espacial que presentan dichas características más simples.

Supongamos, por ejemplo, que el objetivo es detectar caras de un tigre en imágenes

(véase Figura 8). En este caso, las posiciones de los ojos, la boca y la nariz son importantes para realizar una clasificación exitosa. Nuestro objetivo es que, si se realizan pequeños cambios en las ubicaciones de los ojos, nariz y boca (es decir, si tienen una posición diferente), la red sea capaz de realizar la clasificación de la imagen correctamente, gracias a la invarianza respecto a las pequeñas traslaciones de las características.

La **agrupación** es una técnica que consiste en deslizar un filtro (eligiendo un *stride* S) por el mapa de características resultante, C , aplicando una función simple y fija de sus entradas. La capa oculta de la red que aplica dicha técnica recibe el nombre de **capa de agrupación** (en inglés *pooling layer*) y su objetivo es introducir cierta invarianza a pequeñas traslaciones de las características individuales.

Por otro lado, las capas de agrupación también se utilizan para reducir la dimensión del mapa de características, ayudando a evitar un sobreajuste y reduciendo el coste computacional.

El funcionamiento de las capas de agrupación es similar al de las capas convolucionales ya que en ambas se aplica una operación sobre todas las regiones locales del mapa de características, con la diferencia de que la convolución utiliza filtros con pesos entrenables, mientras que la agrupación utiliza una operación fija, sin aprender parámetros.

Las dos variantes más conocidas de agrupación en el ámbito de las redes neuronales son la **agrupación por máximo (max-pooling)** y la **agrupación por promedio (average pooling)**.

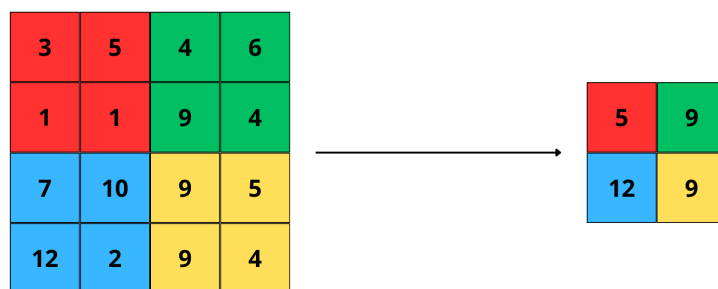


Figura 15: Ilustración del proceso de agrupación por máximo donde se genera un nuevo mapa de características de menor dimensión.

La agrupación por máximo fue introducida por Zhou y Chellappa en el año 1988 [11]. El funcionamiento de esta se basa en el deslizamiento del filtro por el mapa de características aplicando la función máximo, para así extraer las activaciones más representativas. En la Figura 15 podemos observar cómo el tamaño del mapa de características se reduce a la mitad tras la aplicación de la agrupación por máximo, proporcionando invarianza local:

en el momento en que se detecta un valor alto en una región local, la salida asociada a esa región conserva dicho valor alto.

Por otro lado, la agrupación por promedio utiliza la media aritmética de los valores del campo receptivo correspondiente al mapa de características (véase Figura 16).

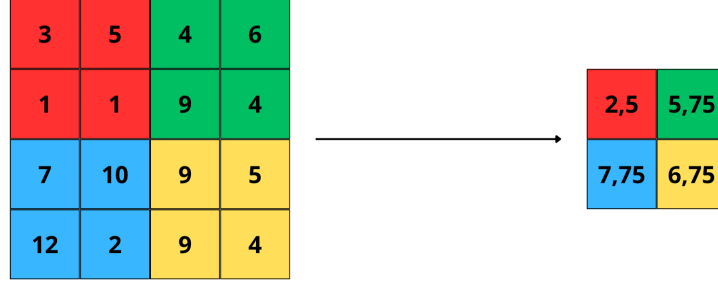


Figura 16: Ilustración del proceso de agrupación por promedio.

Suponiendo que tenemos un mapa de características de dimensión $J \times H$, donde S es el *stride* y $F \times F$ es el tamaño del filtro, asumiendo que no se aplica *padding*, la aplicación de la capa de agrupación generará un mapa de características de dimensión:

$$\left(\left\lfloor \frac{J-F}{S} \right\rfloor + 1 \right) \times \left(\left\lfloor \frac{H-F}{S} \right\rfloor + 1 \right) \quad (57)$$

donde $\lfloor \cdot \rfloor$ es la función parte entera inferior.

Históricamente, la agrupación por promedio, que suaviza las características extraídas, ha sido la más utilizada, pero tras el paso de los años se ha demostrado que la agrupación por máximo es más eficaz y preserva mejor las características más relevantes.

3.6. Convoluciones multidimensionales

A lo largo de toda la sección nos hemos centrado en la arquitectura y las características de las capas ocultas de las redes neuronales cuando la imagen de entrada tiene un único canal (es decir, es una imagen en escala de grises). Por ello, resulta conveniente comentar cómo se extiende toda esta arquitectura al caso de las imágenes con varios canales, es decir, las imágenes en color.

Suponiendo que la imagen de entrada, I , tiene dimensión $J \times H$, al añadirle C_{in} canales para formar una imagen en color, esta se puede representar mediante un **tensor** de dimensión $J \times H \times C_{in}$ (se trata de una generalización de una matriz a tres dimensiones: altura, anchura y canales).

Las imágenes en color se componen de varias matrices superpuestas (véase la Figura 17a) formadas por las intensidades de los píxeles correspondientes a los canales que la forman. Como consecuencia, el campo receptivo de la unidad en una imagen en color estará formado por las intensidades de los píxeles de una región pequeña de cada canal de la imagen, por tanto, tendrá dimensión $M \times M \times C_{in}$, donde $M < \min\{J, H\}$. El conjunto de todos los pesos asociados a los píxeles del campo receptivo definen un filtro de dimensión $M \times M \times C_{in}$, es decir, estará formado por $M^2 C_{in}$ pesos (véase la Figura 17b).

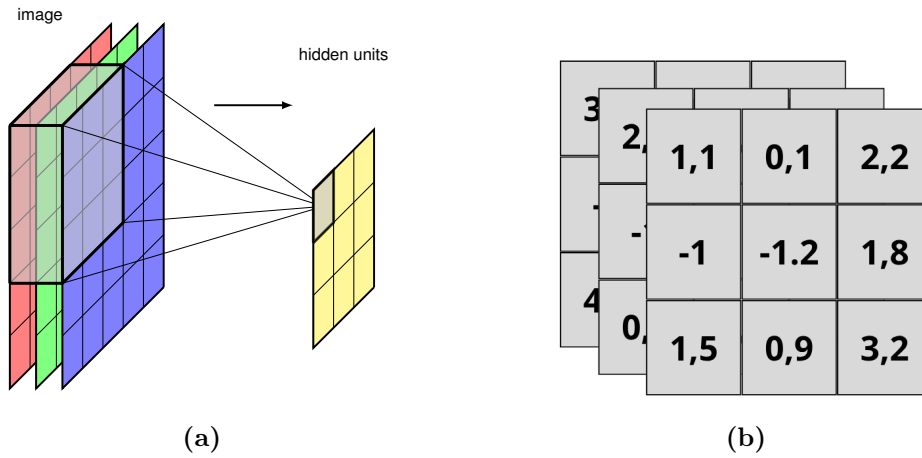


Figura 17: (a) Ilustración del campo receptivo de una imagen de entrada con los canales R, G, B. (b) Ejemplo de un filtro de dimensión $3 \times 3 \times 3$.

Fuente: *Deep Learning: Foundations and Concepts* [1], Figura 10.6 (a), Página 295

Para poder procesar estas imágenes, los filtros de convolución se generalizan y adquieren profundidad para coincidir con el número de canales de entrada. En la práctica, la capa convolucional utiliza múltiples filtros independientes, por tanto, si se aplican C_{out} filtros independientes, cada uno generará un mapa de características distinto. Por tanto, el número de parámetros entrenables de una capa convolucional viene dado por $(M^2 C_{in} + 1) \cdot C_{out}$.

Suponiendo que no hay *padding* y el *stride* es $S = 1$, la dimensión del mapa de características resultante será $(J - M + 1) \times (H - M + 1) \times C_{out}$.

Como conclusión, la convolución multidimensional no introduce ninguna operación nueva, sino que se extiende a las imágenes con varios canales. Por tanto, cada filtro deberá tener la misma profundidad que la imagen de entrada, y con la información de todos los canales se generará un nuevo mapa de características.

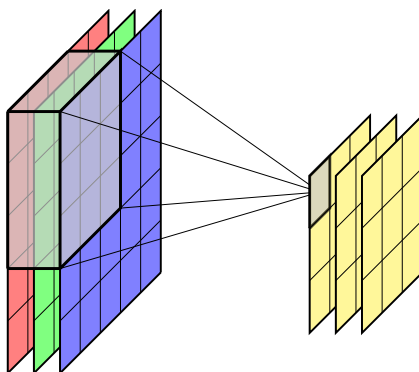


Figura 18: Representación de una capa convolucional con varios filtros independientes. Cada filtro genera un mapa de características y el conjunto de ellos forma la salida.

Fuente: *Deep Learning: Foundations and Concepts* [1], Figura 10.7, Página 296

3.7. Ejemplo práctico de CNN (MNIST)

Para ilustrar las ventajas de las redes neuronales convolucionales y su implementación en un problema de clasificación multiclase, en esta sección estudiaremos el rendimiento de una CNN en el conjunto de datos MNIST introducido en la Sección 2.4.

En particular, consideramos una CNN con 7 capas. Como en la Sección 2.4, la capa de entrada y de salida contienen 784 y 10 unidades respectivamente.

La primera capa oculta es una capa convolucional con un filtro de dimensión 3×3 , *stride* $S = 1$ y *padding* $P = 1$. Esta primera capa es seguida por una capa de agrupación por máximo con un filtro de dimensión 2×2 , *stride* $S = 2$ y *padding* $P = 0$.

La tercera capa oculta es, de nuevo, una capa convolucional con un filtro de dimensión 3×3 , *stride* $S = 1$ y *padding* $P = 1$. De nuevo, la capa convolucional es seguida por una capa de agrupación por máximo, con filtro de dimensión 2×2 , *stride* $S = 2$ y *padding* $P = 0$.

Finalmente, la última capa oculta es una capa completamente conectada, con 256 neuronas. Es decir, cada una de las 256 neuronas recibe un *input* de todas las neuronas de la capa anterior y está conectada a todas las unidades de la capa de salida.

Como en la Sección 2.4, para las neuronas de las capas ocultas (en este caso convolucionales), se utiliza una función de activación *ReLU*, mientras que para las neuronas de la capa de salida se utiliza la función de activación *softmax*. También, como en la Sección 2.4, la función de error utilizada es la entropía cruzada y la métrica para evaluar el rendimiento de la CNN en el conjunto test y la evolución del aprendizaje en los conjuntos de

entrenamiento y validación, la tasa de acierto.

La tasa de acierto de la CNN en el conjunto test es 99,05 %, y por lo tanto, claramente superior a la tasa de la FNN estándar descrita en la Sección 2.4 (97,29 %).

Con respecto al proceso de entrenamiento, en la Figura 19 se muestra la evolución de la tasa de acierto en el conjunto de entrenamiento y de validación como función de la época de entrenamiento.

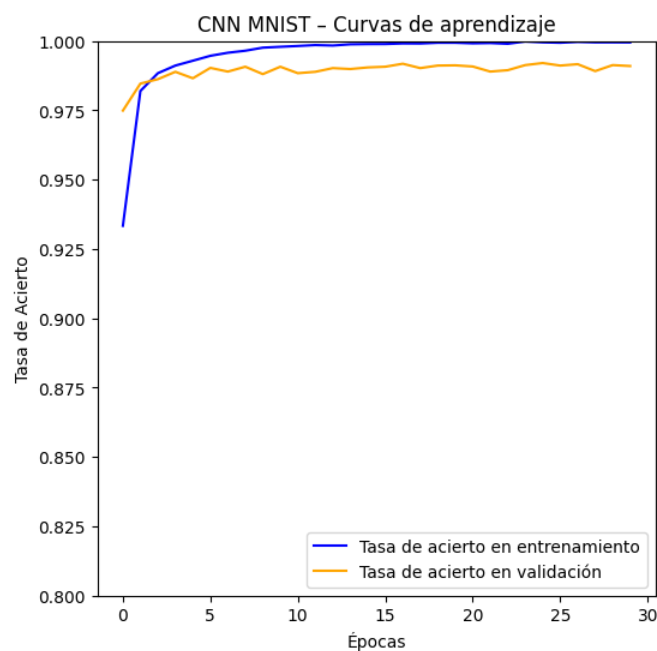


Figura 19: *Curvas de aprendizaje del modelo de CNN descrito.*

Podemos observar cómo desde las primeras épocas, las tasas de acierto en el conjunto de entrenamiento y validación son elevadas. El modelo no solamente aprende correctamente los datos de entrenamiento, sino que generaliza a datos no vistos, es decir, el sobreajuste observado es menor, ya que no hay tanta separación entre las curvas de aprendizaje.

Por último, cabe destacar que, al estabilizarse la tasa de acierto en validación alrededor del 99 % desde las primeras épocas, sería razonable detener el entrenamiento, ya que apenas se obtiene una mejoría del rendimiento del modelo.

Las principales características de la CNN y su rendimiento en el conjunto test se resumen en la Tabla 3.

Red Neuronal Convolutacional	
Arquitectura de Capas	Conv + Pool $\rightarrow$ Conv + Pool $\rightarrow$ 256 $\rightarrow$ 10
Parámetros Totales	824.458
Tasa de Acierto	99,05 %

Tabla 3: Resultados del modelo de CNN implementado.

3.8. Comparación de los modelos implementados

Tras la implementación de los modelos de redes neuronales en las Secciones 2.4 y 3.7, se pueden extraer varias conclusiones.

En la Sección 2.4 se implementó una FNN de una única capa oculta con el conjunto de datos MNIST, obteniendo una tasa de acierto del 97,29 %. Además, se observó en los experimentos realizados que el aumento del número de capas ocultas en este conjunto de datos no proporciona mejores resultados. Por otro lado, la red neuronal convolutacional ha conseguido elevar la tasa de acierto hasta el 99,05 %, obteniendo un rendimiento superior al de la red prealimentada.

Esta mejora que se obtiene con la CNN se debe a que la red aprovecha la estructura espacial de las imágenes de entrada, mientras que en la red prealimentada implementada en la Sección 2.4, los píxeles de la imagen se aplanan en un vector, perdiendo información espacial. La CNN es capaz de detectar características locales de bajo nivel mediante las capas de convolución y de agrupación. Posteriormente, mediante la combinación de estas características de bajo nivel, se obtienen otras más complejas, hasta llegar a la capa de salida donde finalmente la red predice los dígitos que forman el conjunto de datos MNIST.

Por otro lado, el número de parámetros en la red convolutacional ha crecido en comparación con la FNN: en la CNN se han obtenido 824.458 parámetros frente a los 203.530 de la FNN. Sin embargo, en este modelo el coste computacional mayor sí se ve compensado con una mejora en la tasa de acierto del modelo. Además, en la CNN se ha observado un menor sobreajuste en comparación con el modelo FNN implementado en la Sección 2.4.

En conclusión, para el conjunto de datos MNIST, las redes neuronales convolucionales son el modelo más eficaz de los implementados en el trabajo, siendo el modelo elegido para realizar la clasificación de imágenes.

4. Conclusión del trabajo

El objetivo principal de este trabajo ha sido realizar una introducción a las redes neuronales convolucionales y a su aplicación a problemas de clasificación de imágenes.

En primer lugar, se ha presentado la arquitectura de las redes neuronales prealimentadas, comenzando con el modelo más básico, formado por una única capa oculta, hasta generalizar la arquitectura a modelos de redes neuronales prealimentadas multicapa. Además, se ha explicado la base teórica del entrenamiento de las redes neuronales.

En segundo lugar, se ha realizado una explicación teórica de la arquitectura de las redes neuronales convolucionales y se han ilustrado sus ventajas con respecto a redes prealimentadas estándar en un problema de clasificación de imagen multiclase.

Debido a la limitación de la extensión del trabajo como al tiempo de elaboración, existen aspectos relevantes de este trabajo que no se han podido abordar.

Por un lado, sería interesante profundizar en la teoría de las convoluciones multidimensionales y en las técnicas de regularización para evitar un sobreajuste de la red, al igual que en los métodos adaptativos.

Por último, aún centrándonos en el trabajo en el uso de la CNN para problemas de clasificación, también sería interesante abordar otras importantes aplicaciones de las CNN a problemas relacionados con el procesamiento de imágenes que incluyen, entre otras, la detección de objetos y su posición en una imagen, la generación automática de un texto que describe el entorno de la imagen y la generación de imágenes basadas en un *input* de texto que describe el contenido de la misma.

Referencias

- [1] Christopher M. Bishop y Hugh Bishop (2024). *Deep Learning: Foundations and Concepts*. Springer.
- [2] Gareth James, Daniel Witten, Trevor Hastie, Robert Tibshirani y Jonathan Taylor (2023). *An Introduction to Statistical Learning: with Applications in Python*. Springer.
- [3] Aragón, F.J., Goberna, M.A., López, M.A., Rodríguez, M. M. L. (2019). *Nonlinear Optimization*. Springer, Cham.

- [4] Simon Haykin (2009). *Neural Networks and Learning Machines (third edition)*. Editorial Pearson.
- [5] Ian Goodfellow, Yoshua Bengio y Aaron Courville (2016). *Deep Learning*. Editorial MIT Press.
- [6] Ultralytics, Conjunto de datos MNIST. URL: <https://docs.ultralytics.com/es/datasets/classify/mnist/> (Consultado el 26 de Abril de 2026).
- [7] LeCun, Y., Cortes, C. and Burges, C.J.C., The MNIST database of handwritten digits. <http://yann.lecun.com/exdb/mnist/> (Consultado el 26 de Abril de 2026).
- [8] Vincent Dumoulin y Francesco Visin (2016). *A guide to convolution arithmetic for deep learning*. arXiv. <https://arxiv.org/abs/1603.07285>
- [9] Ángel Villazón, Historia de las redes neuronales. <https://www.nuevatribuna.es/articulo/cultura---ocio/historia-redes-neuronales/20250628194552239797.html> (Consultado el 12 de Mayo de 2026).
- [10] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W. y Jackel, L. D. (1989). *Backpropagation Applied to Handwritten Zip Code Recognition*. Neural Computation, 1(4), 541-551.
- [11] Zhou, Y. y Chellappa, R. (1988). *Computation of optic flow using a neural network*. En: *International Conference on Neural Networks*, 71–78. IEEE.
- [12] Suraj Subramanian, Seth Juarez, Cassie Breviu, Dmitry Soshnikov, Ari Bornstein (2021). Learn the Basics <https://docs.pytorch.org/tutorials/beginner/basics/intro.html> (Consultado el 24 de Mayo de 2026).
- [13] John Duchi, Elad Hazan y Yoram Singer (2011). *Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*. Journal of Machine Learning Research, 12, 2121–2159.
- [14] Tijmen Tieleman y Geoffrey Hinton (2012). *Lecture 6.5 – RMSProp: Divide the gradient by a running average of its recent magnitude*. Coursera: *Neural Networks for Machine Learning*, 4(2), 26–31. https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf (Consultado el 27 de Mayo de 2026).
- [15] Diederik P. Kingma y Jimmy Lei Ba (2015). *Adam: A Method for Stochastic Optimization*. arXiv. <https://arxiv.org/pdf/1412.6980>

-
- [16] William Falcon y el equipo de PyTorch Lightning (2019). PyTorch Lightning. Zenodo. DOI: 10.5281/zenodo.3828935. <https://github.com/Lightning-AI/lightning> (Consultado el 26 de Mayo de 2026).

A. Código de Python y figuras empleadas

A través del siguiente enlace se accede al repositorio de GitHub donde se encuentra el código empleado a lo largo de todo el trabajo.

<https://github.com/OscarClimentIvorra/CODIGO-PYTHON-TFG>

A través del siguiente enlace se accede a la carpeta de Canva donde se encuentran todas las figuras elaboradas personalmente y empleadas a lo largo de todo el trabajo

<https://canva.link/loiseo0lxcvko2l>

B. Detalles del desarrollo del trabajo

Tarea	Tiempo (horas)
Recopilación de materiales	15
Estudio de bibliografía	45
Elaboración de resultados gráficos/numéricos	20
Redacción de la memoria	70
Total	150

Tabla 4: *Tiempo aproximado de dedicación al trabajo.*

Asignatura	Páginas	Descripción
Análisis Real de Varias Variables I	19-23	Se utilizan teoremas y conceptos vistos en la asignatura
Probabilidad	11	Se utilizan conceptos vistos en la asignatura
Optimización I	14-23	Se utilizan conceptos vistos en la asignatura
Álgebra lineal I	Todas	Se utilizan conceptos vistos en la asignatura

Tabla 5: *Asignaturas relacionadas con el trabajo.*